

TRƯỜNG ĐẠI HỌC NGOẠI NGỮ - TIN HỌC TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

KHÓA LUẬN TỐT NGHIỆP

**TÌM HIỂU CÁC KỸ THUẬT XỬ LÝ DỮ LIỆU
POINT CLOUD**

GIẢNG VIÊN HƯỚNG DẪN: TÔN QUANG TOẠI

SINH VIÊN THỰC HIỆN: Trần Tấn Kiệt – 22DH111836

Trần Hoàng Phúc - 22DH112862

TP. HỒ CHÍ MINH – 08/2025

TRƯỜNG ĐẠI HỌC NGOẠI NGỮ - TIN HỌC TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

KHÓA LUẬN TỐT NGHIỆP

**TÌM HIỂU CÁC KỸ THUẬT XỬ LÝ DỮ LIỆU
POINT CLOUD**

GIẢNG VIÊN HƯỚNG DẪN: TÔN QUANG TOẠI

SINH VIÊN THỰC HIỆN: Trần Tấn Kiệt – 22DH111836

Trần Hoàng Phúc - 22DH112862

TP. HỒ CHÍ MINH – 08/2025

LỜI CẢM ƠN

Chúng em xin chân thành cảm ơn ThS. Tôn Quang Toại - giảng viên hướng dẫn đã tận tình chỉ dạy và định hướng chúng em trong suốt quá trình thực hiện khóa luận này.

Cảm ơn Khoa Công Nghệ Thông Tin - Trường Đại học Ngoại ngữ - Tin học TP. Hồ Chí Minh đã tạo điều kiện thuận lợi về cơ sở vật chất và môi trường học tập.

Cảm ơn gia đình, bạn bè đã luôn động viên và hỗ trợ chúng em hoàn thành khóa luận.

Cảm ơn cộng đồng nghiên cứu đã chia sẻ các nghiên cứu, bộ dữ liệu KITTI và mô hình PointNet++ làm nền tảng cho đề tài này.

Mặc dù đã nỗ lực, khóa luận vẫn không tránh khỏi thiếu sót. Chúng em mong nhận được góp ý từ thầy cô để hoàn thiện hơn.

LỜI CAM ĐOAN

Chúng em cam đoan khóa luận "TÌM HIỂU CÁC KỸ THUẬT XỬ LÝ DỮ LIỆU POINT CLOUD" là công trình nghiên cứu của riêng chúng em dưới sự hướng dẫn của ThS. Tôn Quang Toại.

Chúng em cam đoan:

1. Toàn bộ nội dung nghiên cứu, thiết kế và cài đặt đều do chúng em tự thực hiện.
2. Các kết quả thực nghiệm là kết quả thực tế từ quá trình chạy mô hình của chúng em.
3. Mã nguồn PointNet++ được tự xây dựng từ đầu, không sao chép từ implementation có sẵn.
4. Tất cả tài liệu tham khảo đều được trích dẫn đầy đủ và chính xác.
5. Khóa luận chưa từng được nộp tại bất kỳ cơ sở giáo dục nào khác.

Nếu có sai phạm, chúng em xin hoàn toàn chịu trách nhiệm theo quy định.

TÓM TẮT KHÓA LUẬN

Bài khóa luận này tập trung nghiên cứu các kỹ thuật xử lý dữ liệu **đám mây điểm** (point cloud) thu được từ cảm biến LiDAR. Trước hết, chúng tôi giới thiệu **bài toán xử lý point cloud** – đây là bài toán quan trọng trong thị giác máy tính 3D với nhiều ứng dụng như xe tự hành, robot và bản đồ số. **Đám mây điểm** là tập hợp các điểm dữ liệu trong không gian ba chiều, mỗi điểm được xác định bởi tọa độ (x, y, z) và có thể kèm thuộc tính như màu sắc, cường độ, thời gian, v.v [1]. Dữ liệu point cloud cung cấp thông tin hình học phong phú giúp nhận dạng vật thể và tái tạo hiện trường, nhưng **không có cấu trúc lưới** cố định, gây thách thức cho việc xử lý [1].

Khóa luận tiến hành **tổng quan các kỹ thuật xử lý point cloud truyền thống**. Chúng tôi tìm hiểu nguyên lý hoạt động của cảm biến **LiDAR (Light Detection and Ranging)** – công nghệ phát ra các xung laser và đo thời gian phản xạ để xây dựng bản đồ 3D của môi trường [2]. LiDAR thu thập hàng triệu điểm mỗi giây, tạo nên các đám mây điểm 3D có độ chính xác cao. Chúng tôi cũng trình bày các bước tiền xử lý point cloud phổ biến như **lấy mẫu (downsampling)** nhằm giảm số lượng điểm nhưng vẫn bảo toàn hình dạng chính, các phương pháp **lọc nhiễu** gồm loại bỏ điểm ngoài (radius/statistical outlier) để loại các điểm cô lập bất thường, tính **pháp tuyến** bề mặt cho mỗi điểm, và cấu trúc **cây k-d** để tăng tốc tìm kiếm láng giềng gần [3]. Bên cạnh đó, khóa luận điểm qua thuật toán **ICP (Iterative Closest Point)** – giải pháp kinh điển để căn chỉnh hai đám mây điểm bằng cách lặp tìm cặp điểm tương ứng và cập nhật phép biến đổi hình học [4].

Tiếp theo, khóa luận **tổng quan các phương pháp học sâu xử lý point cloud**. Chúng tôi phân loại các hướng tiếp cận chính: (1) **Phương pháp dựa trên điểm trực tiếp (point-based)** – tiêu biểu như PointNet và PointNet++ – học đặc trưng trực tiếp trên tọa độ các điểm và trích xuất đặc trưng cục bộ qua kiến trúc phân cấp [5]. (2) **Phương pháp dựa trên voxel (voxel-based)** – chuyển point cloud thành lưới **voxel** (khối điểm 3D tương tự pixel 2D) rồi áp dụng mạng tích chập 3D [6]; ví dụ: VoxelNet chia không gian thành các voxel đều đặn và dùng tích chập 3D để

nhận dạng đối tượng [8]. (3) **Phương pháp Pillar (voxel 2,5D)** – dựa trên chiếu điểm xuống mặt phẳng ngang (bird’s eye view), chẳng hạn thuật toán PointPillars coi mỗi cột dọc (pillar) như 1 voxel 2D để xử lý bằng tích chập 2D [6]. (4) **Phương pháp chiếu ảnh (projection-based)** – chiếu point cloud lên mặt phẳng 2D (theo mặt cầu hoặc mặt phẳng) rồi dùng mạng CNN truyền thống; tiêu biểu là SqueezeSeg, RangeNet++ chiếu lên ảnh phạm vi và đạt tốc độ cao nhưng đánh đổi bởi mất mát thông tin khi chiếu [5]. (5) **Phương pháp dựa trên đồ thị (graph-based)** – xem các điểm như nút của đồ thị và áp dụng mạng nơ-ron đồ thị để trao đổi thông tin lân cận, ví dụ: DGCNN. (6) **Phương pháp lai (hybrid)** – kết hợp ưu điểm của nhiều phương pháp, chẳng hạn PV-RCNN kết hợp cả voxel và point để tăng hiệu quả [6].

Giải pháp đề xuất của khóa luận là áp dụng mô hình học sâu PointNet++ cho bài toán phân tích point cloud. PointNet++ mở rộng từ PointNet, khắc phục nhược điểm không tận dụng cấu trúc cục bộ của PointNet bằng cách chia điểm thành các cụm lân cận ở nhiều độ lớn khác nhau và áp dụng PointNet trên từng cụm (set abstraction)[5] [7]. Mô hình kiến trúc PointNet++ gồm ba thành phần chính lặp lại: **tầng lấy mẫu (sampling layer)** chọn ra các điểm trung tâm (centroid) đại diện cho cụm điểm địa phương (thường dùng giải thuật Farthest Point Sampling để chọn các điểm cách xa nhất đảm bảo phủ không gian đều đặn [7]); **tầng nhóm (grouping layer)** thu thập các điểm láng giềng xung quanh mỗi centroid (dựa trên bán kính hoặc k láng giềng gần nhất) để hình thành tiểu cụm điểm [7]; và **tầng PointNet cục bộ** áp dụng một mạng PointNet nhỏ (MLP + pooling) để học vector đặc trưng cố định cho từng cụm điểm địa phương [7]. Kiến trúc phân cấp này có thể mở rộng nhiều mức: đặc trưng cục bộ từ mức dưới được tổng hợp thành đặc trưng cấp cao hơn qua Multi-Scale Grouping và Multi-Resolution Grouping [7]. Nhờ đó, PointNet++ thu được cả đặc trưng cục bộ lẫn ngữ cảnh toàn cục, cải thiện độ chính xác đáng kể so với PointNet gốc [5] [7]. Mô hình PointNet++ trong khóa luận được cài đặt dựa trên mã nguồn PyTorch của Qi và cs. (2017) và điều chỉnh cho các tác vụ phân loại, phân đoạn và phát hiện đối tượng 3D.

Chúng tôi đã **thực nghiệm mô hình PointNet++** trên các tác vụ cụ thể. Thứ nhất, với **phân loại vật thể 3D** trên tập dữ liệu KITTI (trích xuất các cụm điểm tương ứng với các đối tượng như xe hơi, người đi bộ, xe đạp), mô hình đạt độ chính xác phân loại cao $\sim 91\%$ (tỷ lệ dự đoán đúng) trên tập kiểm thử. Thứ hai, với **phân đoạn ngữ nghĩa** trên bộ dữ liệu **Semantic-KITTI** (gồm các đám mây điểm LiDAR gán nhãn điểm-theo-điểm 19 lớp như xe, người, đường, cây cối...), mô hình PointNet++ đạt kết quả khả quan: độ chính xác chung $\sim 79\%$ và **mIoU** (độ giao phủ trung bình) $\sim 62,29\%$ sau 5 epoch huấn luyện.

Kết quả đạt được chứng tỏ **hiệu quả của PointNet++** trong việc xử lý dữ liệu point cloud: mô hình có khả năng phân loại chính xác đối tượng 3D và phân đoạn điểm theo ngữ cảnh tốt trên dữ liệu thực tế (LiDAR gắn trên xe tự hành). Tuy nhiên, mô hình vẫn có hạn chế về thời gian suy luận và độ chính xác khi so sánh với các phương pháp tối tân hơn (ví dụ: Transformer 3D hoặc mạng tích chập sparse 3D). Khóa luận cũng đề xuất một số hướng cải tiến mô hình (như thay thế MLP truyền thống bằng cơ chế attention, hoặc kết hợp đầu ra đa thang đo) để nâng cao hiệu năng xử lý point cloud trong tương lai.

Từ khóa: LiDAR, đám mây điểm, PointNet++, phân đoạn 3D, phân loại 3D, phát hiện đối tượng 3D, xe tự hành, học sâu 3D.

MỤC LỤC

Chương 1: Giới thiệu bài toán

1.1 Câu hỏi nghiên cứu.....	10
1.2 Giới thiệu Dataset được sử dụng để giải quyết bài toán.....	12
1.3 Các phương pháp đánh giá bài toán.....	14
1.3.1 Ma trận nhầm lẫn (Confusion matrix).....	14
1.3.2 Độ chính xác (Accuracy).....	14
1.3.3 Độ chính xác theo Precision.....	15
1.3.4 Độ nhạy (Recall).....	16
1.3.5 F1-Score.....	16
1.3.6 Intersection over Union (IoU).....	17
1.3.7 Average Precision (AP).....	17
1.3.8 mean Average Precision (mAP).....	18
1.4 Tầm quan trọng.....	18
1.4.1 Tính mới.....	18
1.4.2 Tính hữu dụng.....	20
1.5 Giới hạn nghiên cứu.....	21
1.6 Bố cục khóa luận.....	23

Chương 2: Lý thuyết nền tảng

2.1 Công nghệ LiDAR và dữ liệu đám mây điểm.....	27
2.2 Các kỹ thuật xử lý đám mây điểm cơ bản.....	29
2.3 Kỹ thuật nâng cao: Đăng ký điểm (Point Cloud Registration).....	32
2.4 Deep Learning trên dữ liệu point cloud.....	33

Chương 3: Các công trình liên quan

3.1 Phương pháp xử lý point cloud truyền thống (classical).....	36
3.2 Mô hình học sâu cho Point Cloud.....	38
3.3 PointNet và PointNet++.....	41

Chương 4: Giải pháp đề xuất

4.1 Phát biểu tổng quan bài toán.....	45
4.2 Phân tích dữ liệu.....	46
4.3 Mô hình.....	50
4.4 Thực nghiệm.....	56
4.4.1 Thực nghiệm 1: Phân lớp đối tượng trên point cloud KITTI.....	56
4.4.2 Thực nghiệm 2: Phân đoạn ngữ nghĩa Semantic-KITTI.....	58

Chương 5: Kết luận

5.1 Những điều đã làm được.....	64
5.2 Những điều chưa làm được.....	65
5.3 Hướng phát triển.....	67

DANH MỤC CÁC KÝ HIỆU, CHỮ VIẾT TẮT

STT	KÝ HIỆU, CHỮ VIẾT TẮT	Ý NGHĨA
1	LiDAR	Light Detection and Ranging – Cảm biến quét laser thu thập khoảng cách.
2	GNSS	Global Navigation Satellite System – Hệ thống định vị vệ tinh toàn cầu (ví dụ: GPS).
3	IMU	Inertial Measurement Unit – Bộ đo quán tính (cảm biến gia tốc + con quay).
4	Point Cloud	Đám mây điểm – Tập hợp các điểm dữ liệu 3D trong không gian.
5	Voxel	Phần tử thể tích 3 chiều (tương tự pixel trong 2D) dùng để lượng tử hóa không gian.
6	FPS	Farthest Point Sampling – Thuật toán chọn điểm xa nhất (kỹ thuật lấy mẫu điểm cloud).
7	MLP	Multilayer Perceptron – Mạng nơ-ron truyền thẳng nhiều tầng (thường dùng để học đặc trưng điểm).
8	CNN	Convolutional Neural Network – Mạng nơ-ron tích chập (phổ biến trong xử lý ảnh 2D).
9	IoU	Intersection over Union – Độ giao phủ trên hợp, dùng để đánh giá vùng dự đoán và thực tế.

10	AP	Average Precision – Độ chính xác trung bình (diện tích dưới đường P-R) cho một lớp.
11	mAP	mean Average Precision – Trung bình AP trên các lớp (đánh giá chung mô hình).
12	mIoU	mean Intersection over Union – Trung bình IoU trên các lớp (thước đo cho phân đoạn).
13	ADAS	Advanced Driver-Assistance Systems – Hệ thống hỗ trợ lái xe nâng cao (ứng dụng công nghệ cảm biến như radar, LiDAR).
14	KITTI	Bộ dữ liệu xe tự hành do Karlsruhe Institute of Technology & TUM phát hành (gồm dữ liệu camera, LiDAR với nhãn thực tế).
15	ROS	Robot Operating System – Hệ điều hành và middleware cho robot (hỗ trợ thao tác dữ liệu cảm biến như LiDAR).

DANH MỤC CÁC BẢNG

STT	BẢNG
1	Bảng 4.2.1 Phân bố số lượng điểm theo từng lớp của bộ dữ liệu Semantic-KITTI.
2	Bảng 4.2.2 Kiến trúc mạng PointNet++ chi tiết các tầng và tham số.

DANH MỤC CÁC HÌNH VẼ, BIỂU ĐỒ

STT	HÌNH VẼ, BIỂU ĐỒ
1	Hình 1.1 – Mô phỏng quét LiDAR xung quanh một xe tự hành, tạo ra đám mây điểm 3D của môi trường [2].
2	Hình 2.1: Cách thức cũng như công thức tính toán trên LIDAR
3	Hình 3.1: Kiến trúc PointNet. Mạng phân loại lấy n điểm làm đầu vào, áp dụng các phép biến đổi đầu vào và đặc trưng, sau đó tổng hợp các đặc trưng điểm bằng phương pháp gộp tối đa (max pooling). Đầu ra là điểm phân loại cho m lớp. Mạng phân đoạn là một phần mở rộng của mạng phân loại. Nó nối các đặc trưng toàn cục và cục bộ, đồng thời xuất ra điểm theo từng điểm. MLP là viết tắt của perceptron đa lớp, các số trong ngoặc đơn là kích thước lớp của nó. Batchnorm được sử dụng cho tất cả các lớp với ReLU. Các lớp dropout được sử dụng cho MLP cuối cùng trong mạng phân loại. (nguồn PointNet)
4	Hình 4.2 - Kiến trúc PointNet++
5	Hình 4.3: Quá trình huấn luyện mô hình phân lớp PointNet++ – biểu đồ loss và accuracy trên tập huấn luyện (màu xanh) và tập validation (màu đỏ).
6	Hình 4.4: Kết quả phân đoạn ngữ nghĩa trên một frame LiDAR. Hình trái là ground truth (mỗi màu tương ứng một nhãn: xanh dương – xe, đỏ – người, nâu – mặt đường, v.v.), hình phải là kết quả mô hình PointNet++ dự đoán. Mô hình phân đoạn khá tốt các vùng lớn như đường và xe, nhưng bỏ sót một vài người đi bộ (khoanh tròn).

7	Hình 4.5: Ví dụ kết quả phát hiện 3D bằng PointNet++. Màu xanh: hộp 3D ground truth; Màu đỏ: hộp 3D do mô hình dự đoán. Trường hợp bên trái là một xe hơi được phát hiện đúng loại nhưng hộp đỏ hơi lệch vị trí; bên phải là người đi bộ được phát hiện (mô hình khoanh vùng đúng người, nhưng hộp đỏ lớn hơn thực tế).
---	--

Chương 1. Giới thiệu bài toán

1.1 Câu hỏi nghiên cứu

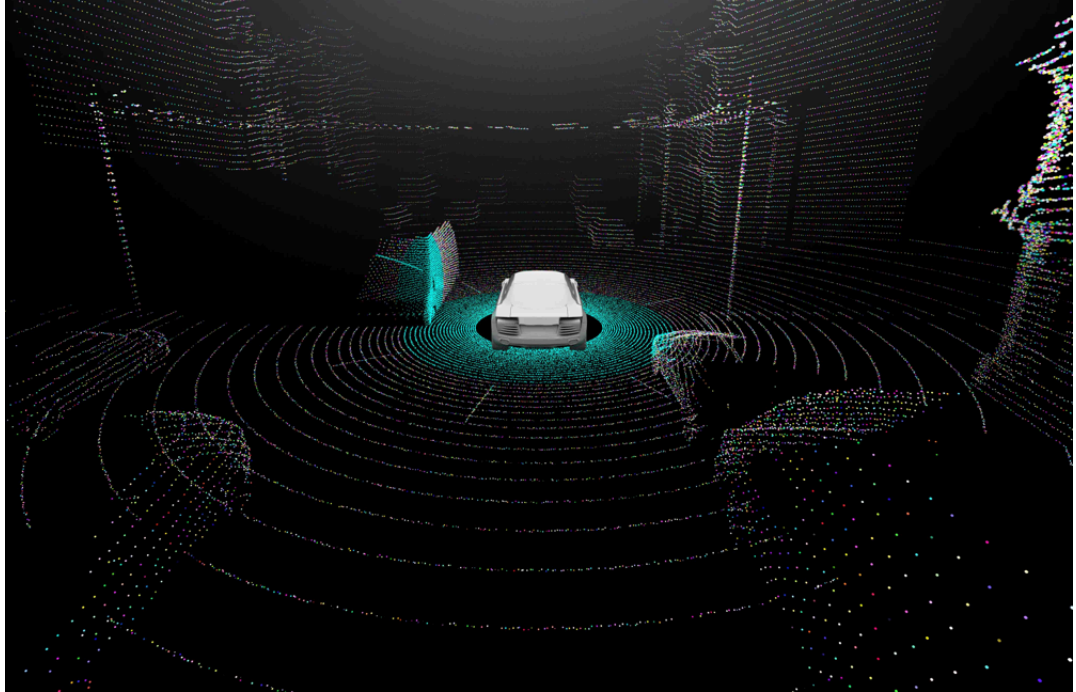
Phát biểu bài toán: Bài toán nghiên cứu đặt ra là xử lý và phân tích dữ liệu đám mây điểm (point cloud) thu được từ cảm biến LiDAR. Cụ thể, làm thế nào để từ dữ liệu point cloud thô (tập hợp hàng triệu điểm rời rạc trong không gian 3D) có thể **nhận diện và hiểu được các đối tượng và cấu trúc trong cảnh**. Bài toán này bao gồm các tác vụ con như: phân loại đối tượng 3D (nhận biết loại vật thể từ hình dạng point cloud của nó), phân đoạn ngữ nghĩa (gán nhãn cho từng điểm theo loại bề mặt/vật thể), và phát hiện đối tượng 3D (xác định vị trí và kích thước các đối tượng cụ thể như xe cộ, người đi bộ trong đám mây điểm).

Đầu vào: Là **đám mây điểm 3D** thu được từ cảm biến LiDAR gắn trên xe tự hành hoặc thiết bị quét laser. Mỗi point cloud thường ở dạng tập hợp điểm trong không gian tọa độ thời gian thực của cảm biến (hoặc trong hệ tọa độ địa phương của xe tự hành). Dữ liệu có thể kèm thuộc tính khác như cường độ phản xạ laser của từng điểm.

Đầu ra mong muốn: Kết quả của hệ thống xử lý point cloud có thể ở các dạng tùy bài toán cụ thể:

- Đối với bài toán phân loại, đầu ra là **nhãn lớp** của toàn bộ point cloud đầu vào (ví dụ: xác định đám mây điểm này thuộc về loại đối tượng nào – ô tô, cây cối hay người,...).
- Đối với bài toán phân đoạn, đầu ra là **nhãn ngữ nghĩa cho từng điểm** trong point cloud (ví dụ: điểm nào thuộc mặt đường, điểm nào thuộc người đi bộ, xe hơi, tòa nhà...).

- Đối với bài toán phát hiện 3D, đầu ra là danh sách các **hộp giới hạn 3D** (bounding box) hoặc các cụm điểm tương ứng với từng đối tượng được phát hiện, kèm nhãn loại đối tượng và độ tin cậy.



Hình 1.1 – Mô phỏng quét LiDAR xung quanh một xe tự hành, tạo ra đám mây điểm 3D của môi trường. (nguồn [Xe tự hành](#))

Ví dụ minh họa: Hình 1.1 ở trên là ví dụ về đầu vào và đầu ra dạng point cloud. Đầu vào là đám mây điểm thu được khi xe tự hành quét laser môi trường xung quanh – ta thấy hàng trăm nghìn điểm (màu trắng) biểu diễn các bề mặt đường, vỉa hè, tòa nhà và vật thể (xe hơi khác) xung quanh xe. Yêu cầu đầu ra có thể là: xác định cụm điểm tạo thành chiếc xe màu xanh bên trái là một xe ô tô (phát hiện + phân loại), phân biệt các điểm thuộc mặt đường (màu khác) với điểm thuộc người đi bộ, cây cối, v.v (phân đoạn). Chẳng hạn, hệ thống cần nhận ra các **cụm điểm hình hộp** phía trước là các xe ô tô khác và khoanh vùng chúng, hoặc gán màu nhãn (đỏ, xanh...) cho các điểm tương ứng từng loại đối tượng.

Tóm lại, bài toán đặt ra câu hỏi: Làm thế nào để máy tính có thể “nhìn thấy” thế giới thực qua dữ liệu point cloud, phân biệt và nhận dạng được các đối tượng và bề mặt giống như con người?

1.2 Giới thiệu Dataset được sử dụng để giải quyết bài toán

Để giải quyết bài toán, khóa luận sử dụng bộ dữ liệu point cloud thu thập từ cảm biến LiDAR gắn trên xe tự hành. Cụ thể, chúng tôi lựa chọn **bộ dữ liệu KITTI** – đây là bộ dữ liệu tiêu chuẩn trong lĩnh vực xe tự lái, cung cấp cả dữ liệu camera và LiDAR cho hàng chục ngàn khung cảnh lái xe thực tế [5]. Trong phạm vi khóa luận, chúng tôi tập trung vào dữ liệu LiDAR của KITTI và phân nhánh thành hai bộ con phục vụ hai tác vụ khác nhau:

- **Semantic-KITTI:** Bộ dữ liệu Semantic-KITTI [5] mở rộng từ KITTI, cung cấp nhãn ngữ nghĩa điểm cho toàn bộ 43,552 frame quét LiDAR (ứng với 21 đoạn đường, ~43K đám mây điểm) [5]. Mỗi điểm trong đám mây được gán một trong 28 nhãn vật thể/địa vật (19 lớp chính sử dụng để đánh giá, bao gồm: bề mặt đường, vỉa hè, tòa nhà, hàng rào, cột tín hiệu, biển báo, người đi bộ, người đi xe đạp, xe hơi, xe tải, xe buýt, xe máy, cây cối, thảm thực vật, v.v.). Dữ liệu Semantic-KITTI có độ phong phú cao, mỗi frame chứa ~120k điểm, bao quát nhiều tình huống giao thông ngoại ô (tuy nhiên chưa đa dạng môi trường thành phố đông đúc) [5]. Bộ dữ liệu này được dùng để huấn luyện và đánh giá thuật toán **phân đoạn điểm** trong khóa luận. Chúng tôi chia tập dữ liệu theo khuyến nghị: sử dụng các sequence 00-10 (trừ 08) cho huấn luyện, sequence 08 để kiểm thử (theo chuẩn benchmark Semantic-KITTI).
- **KITTI Object Detection:** Đây là phần dữ liệu KITTI được gán nhãn hộp 3D cho đối tượng (xe hơi, người đi bộ, người đi xe đạp) trong 7,481 ảnh khung

cảnh đào tạo [8]. Tương ứng mỗi ảnh có một đám mây điểm LiDAR và file nhãn chứa các hộp 3D (tọa độ, kích thước và loại đối tượng). Chúng tôi sử dụng tập này để xây dựng bộ dữ liệu **phân loại và phát hiện đối tượng**. Cách chuẩn bị dữ liệu: (i) Trích xuất các cụm điểm của từng đối tượng từ khung LiDAR dựa trên hộp 3D nhãn, tạo tập point cloud nhỏ cho từng đối tượng (dùng trong bài toán phân loại 3 lớp: Car, Pedestrian, Cyclist). (ii) Sử dụng toàn bộ khung LiDAR kèm nhãn hộp 3D để huấn luyện mô hình phát hiện 3D (dự đoán hộp 3D quanh đối tượng trong cloud). Trong quá trình thực nghiệm, chúng tôi chia ngẫu nhiên $\sim 70\%$ khung để huấn luyện, $\sim 10\%$ để xác thực và $\sim 20\%$ để kiểm thử mô hình, đảm bảo không trùng lặp cảnh giữa các tập.

Đặc điểm dữ liệu: Dữ liệu LiDAR KITTI có các đặc trưng: tần suất quét ~ 10 Hz, mỗi quét $\sim 100k$ điểm từ cảm biến Velodyne HDL-64E (64 tia laser) [5]. Phạm vi quét 360° quanh xe, tầm xa ~ 120 m, độ phân giải góc dọc $\sim 0.4^\circ$. Dữ liệu tương đối thưa (mật độ điểm giảm theo khoảng cách) và **không có cấu trúc lưới** – các điểm phân bố không đều, vùng xa xe điểm rất thưa thớt. Điều này ảnh hưởng đến độ khó bài toán: mô hình phải chịu được dữ liệu không đồng nhất (đặc biệt trong phân đoạn).

Tiền xử lý dữ liệu: Trước khi đưa vào mô hình, point cloud được tiền xử lý: (i) **Cắt bỏ** những điểm quá xa hoặc không quan trọng (ví dụ loại điểm trời, điểm mặt đất xa $> 50m$ nếu không cần thiết) nhằm giảm nhiễu; (ii) **Lấy mẫu (downsample)** đám mây điểm dùng kỹ thuật voxel grid – chia không gian thành lưới lập phương cạnh 0.1 m, mỗi ô lập phương lấy trung bình điểm – để giảm số điểm xử lý [3]; (iii) **Chuẩn hóa tọa độ** về gốc tọa độ xe hoặc trích đặc trưng (vd. tọa độ tương đối so với xe). Ngoài ra với tác vụ phân loại, ta cân bằng số điểm mỗi mẫu (ví dụ trích đúng 1024 điểm mỗi đối tượng bằng cách lấy mẫu ngẫu nhiên nếu vật thể có nhiều

điểm hoặc lặp lại nếu ít điểm). Những bước này đảm bảo dữ liệu đầu vào nhất quán về định dạng cho mô hình học sâu.

Tóm lại, **bộ dữ liệu KITTI và Semantic-KITTI** cung cấp nền tảng để nghiên cứu: chúng đa dạng về cảnh thực tế, có nhãn đầy đủ và đã được cộng đồng sử dụng rộng rãi, giúp chúng tôi có thước đo so sánh kết quả với các phương pháp khác.

1.3 Các phương pháp đánh giá bài toán

Để đánh giá hiệu quả của mô hình trong các tác vụ xử lý point cloud, khóa luận sử dụng các **chỉ số hiệu năng chuẩn** trong lĩnh vực thị giác máy tính và học máy, tuân theo chuẩn đánh giá của bộ dữ liệu KITTI. Dưới đây là các phương pháp và chỉ số đánh giá chính:

1.3.1 Ma trận nhầm lẫn (Confusion matrix)

Ma trận nhầm lẫn là bảng (với bài toán nhị phân) tổng kết số lượng dự đoán đúng và sai của mô hình trên tập kiểm thử [9]. Đối với từng lớp, ma trận ghi lại: True Positives (TP) – số mẫu thuộc lớp đó được dự đoán đúng; False Positives (FP) – số mẫu không thuộc lớp đó nhưng bị nhầm lẫn thành lớp đó; False Negatives (FN) – số mẫu thuộc lớp đó nhưng mô hình dự đoán sai thành lớp khác; và True Negatives (TN) – số mẫu không thuộc lớp đó và được dự đoán đúng là không phải. Ma trận nhầm lẫn cung cấp cái nhìn chi tiết về loại lỗi mô hình hay mắc phải (nhầm lẫn giữa những cặp lớp nào).

1.3.2 Độ chính xác (Accuracy)

Accuracy đo lường tỷ lệ dự đoán đúng trên tổng số dự đoán [9]. Đây là chỉ số đơn giản:

$$\text{Accuracy} = \frac{\text{Số lượng mẫu dự đoán đúng}}{\text{Tổng số mẫu}} = \frac{TP + TN}{TP + FP + FN + TN} \quad (1.1)$$

Trong đó :

- **TP** (True Positive): Dự đoán đúng cho lớp dương
- **TN** (True Negative): Dự đoán đúng cho lớp âm
- **FP** (False Positive): Dự đoán sai (dự đoán dương nhưng thực tế âm)
- **FN** (False Negative): Dự đoán sai (dự đoán âm nhưng thực tế dương)

Ví dụ, trong phân loại 3D 3 lớp (Car/Pedestrian/Cyclist), accuracy là phần trăm số đối tượng được gán đúng loại. Mặc dù độ chính xác trực quan dễ hiểu, nhưng **trong trường hợp dữ liệu mất cân bằng** (ví dụ 90% mẫu là Car, 5% Pedestrian, 5% Cyclist), accuracy có thể cao ngay cả khi mô hình chỉ dự đoán luôn là “Car”. Do đó, cần kết hợp với các chỉ số khác để đánh giá công bằng hơn khi dữ liệu lệch.

1.3.3 Độ chính xác theo Precision

Precision cho biết trong các mẫu mô hình dự đoán là dương (thuộc một lớp nhất định), có bao nhiêu phần trăm thật sự đúng là lớp đó [\[10\]](#).

Công thức:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (1.2)$$

Trong đó:

- **Tử số (TP)**: Số mẫu dự đoán đúng là dương
- **Mẫu số (TP + FP)**: Tổng số mẫu được dự đoán là dương (bao gồm cả đúng và sai)

Precision cao nghĩa là ít dự đoán nhầm sang lớp dương khi thực tế không phải. Chỉ số này đặc biệt quan trọng trong phát hiện vật thể: ví dụ precision của lớp “Người đi bộ” = 80% nghĩa là trong 100 dự đoán “có người đi bộ”, có 80 dự đoán đúng (20 dự

đoán nhầm). Precision giúp đánh giá **độ tin cậy của các dự đoán dương** – mô hình có “báo động giả” nhiều hay không.

1.3.4 Độ nhạy (Recall)

Recall (hay Sensitivity) cho biết trong các mẫu thực sự thuộc lớp dương, mô hình tìm ra được bao nhiêu phần trăm [9].

Công thức:

$$\text{Recall} = \frac{TP}{TP + FN}$$

(1.3)

Trong đó:

- **Tử số (TP):** Số mẫu dự đoán đúng là dương
- **Mẫu số (TP + FN):** Tổng số mẫu thực sự là dương (bao gồm cả được tìm ra và bị bỏ sót)

Recall cao nghĩa là mô hình **bắt được hầu hết các trường hợp dương**, ít bỏ sót. Trong phát hiện vật thể 3D, recall lớp “Xe hơi” = 90% nghĩa là trong 100 xe hơi thực tế, mô hình phát hiện được 90, bỏ lỡ 10. Thường có sự đánh đổi: mô hình tăng recall (bắt nhiều đối tượng hơn) thì có thể giảm precision (bắt nhầm nhiều hơn). Do đó thường xét đồng thời precision và recall.

1.3.5 F1-Score

F1-score là **trung bình điều hòa** của Precision và Recall [9].

Công thức:

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

(1.4)

Chỉ số này tổng hợp một con số duy nhất cân bằng giữa precision và recall, hữu ích khi so sánh mô hình tổng quát. F1 đạt giá trị cao chỉ khi cả precision và recall đều cao, do đó F1 giảm nếu mô hình quá thiên lệch (ví dụ precision cao nhưng recall thấp). Trong đánh giá phân đoạn Semantic-KITTI, người ta cũng có thể dùng F1-score cho từng lớp để cân bằng giữa lỗi bỏ sót và nhận nhầm trên lớp đó.

1.3.6 Intersection over Union (IoU)

IoU đo độ trùng khớp giữa vùng dự đoán và vùng thực tế, thường dùng cho bài toán phân đoạn và phát hiện với vùng/bounding box.

Công thức:

$$\text{IoU} = \frac{\text{Diện tích hoặc số điểm giao giữa dự đoán và Ground Truth}}{\text{Diện tích hợp lại giữa dự đoán và Ground Truth}} \quad (1.5)$$

Trong phân đoạn điểm, IoU của lớp k sẽ được tính theo công thức:

$$\text{IoU}_k = \frac{\text{Số điểm lớp k đúng với label}}{\text{số điểm thuộc lớp k (GT) + số điểm dự đoán là k - điểm đúng label}} \quad (1.6)$$

IoU = 1 là trùng khớp hoàn hảo, IoU = 0 nghĩa là không trùng khớp gì. Chỉ số IoU đánh giá trực quan chất lượng phân đoạn từng lớp hoặc chất lượng hộp dự đoán trong phát hiện (IoU giữa hộp 3D dự đoán và hộp ground truth).

1.3.7 Average Precision (AP)

AP là chỉ số tổng hợp cho bài toán phát hiện (hoặc phân đoạn nhiều lớp) trên cơ sở đường **Precision-Recall**. Cụ thể, AP được tính bằng diện tích dưới đường cong Precision-Recall của mô hình trên một lớp đối tượng nhất định [9]. Thông thường, để tính AP, người ta sắp xếp các dự đoán theo độ tự tin giảm dần, sau đó quét ngưỡng và tính Precision, Recall tại mỗi ngưỡng đó, rồi lấy tích phân (hay tổng

dưới dạng rời rạc 11 điểm) của Precision theo Recall [10]. AP phản ánh khả năng phát hiện đối tượng của mô hình trên toàn dải độ nhạy mô hình hoàn hảo có $AP = 100\%$, mô hình ngẫu nhiên cho AP thấp. KITTI đánh giá phát hiện 3D bằng AP trên các đối tượng Car, Pedestrian, Cyclist tại các ngưỡng IoU nhất định (ví dụ $IoU > 0.7$ với Car, > 0.5 với Pedestrian).

1.3.8 mean Average Precision (mAP)

mAP là trung bình của AP trên các lớp hoặc trên các ngưỡng đánh giá khác nhau. Trong KITTI 3D detection, mAP thường hiểu là trung bình AP của cả 3 lớp chính hoặc theo độ khó. Còn trong MS COCO, mAP tính trung bình AP trên 10 ngưỡng IoU khác nhau [10]. Nói chung, mAP cung cấp **một con số tổng quát** thể hiện hiệu năng mô hình trên toàn bộ các lớp đối tượng [10]. Ví dụ, $mAP = 70\%$ nghĩa là trung bình mô hình đạt 70% AP trên các lớp – giúp so sánh tổng thể các mô hình một cách khách quan.

Tóm lại, trong khóa luận, chúng tôi sử dụng các chỉ số trên để đánh giá mô hình PointNet++ huấn luyện: **Accuracy** cho bài toán phân loại (đối chiếu với kết quả phân loại truyền thống), **IoU** và **mIoU** cho bài toán phân đoạn từng lớp và trung bình [7], và **AP**, **mAP** cho bài toán phát hiện 3D theo chuẩn KITTI. Những chỉ số này đảm bảo việc đánh giá bám sát tiêu chuẩn học thuật và thực tiễn, cho phép so sánh với các phương pháp khác một cách rõ ràng.

1.4 Tầm quan trọng

1.4.1 Tính mới

Đề tài “Tìm hiểu các kỹ thuật xử lý dữ liệu Point Cloud” mang tính mới ở chỗ đây là một hướng nghiên cứu hiện đại và đang phát triển nhanh trong lĩnh vực thị giác máy tính 3D. Các điểm mới có thể thấy gồm:

Dữ liệu mới mẻ: Thay vì ảnh 2D truyền thống, đề tài tập trung vào dữ liệu point cloud 3D – dạng dữ liệu cảm biến mới trở nên phổ biến nhờ sự bùng nổ của công nghệ LiDAR. Xử lý trực tiếp trên đám mây điểm là hướng tiếp cận mới so với việc phải chuyển về ảnh 2D hay lưới 3D trước kia.

- Phương pháp mới: Đề tài tổng hợp và áp dụng những kỹ thuật tiên tiến nhất dành cho point cloud, đặc biệt là mô hình học sâu PointNet++. Đây là mô hình mới (xuất hiện 2017) nhưng đã trở thành nền tảng cho nhiều nghiên cứu sau đó. Chúng tôi không dừng ở lý thuyết mà trực tiếp cài đặt, huấn luyện mô hình này trên dữ liệu thực, từ đó nắm bắt ưu nhược điểm và khả năng ứng dụng. Việc triển khai PointNet++ trên Semantic-KITTI cũng chưa nhiều nghiên cứu trong nước thực hiện.
- Kết hợp nhiều khía cạnh: Khóa luận bao quát cả xử lý truyền thống (downsample, ICP, ...) lẫn học sâu hiện đại trên point cloud. Việc tích hợp hai mảng kiến thức này tạo góc nhìn toàn diện, có thể đề xuất cải tiến mới. Ví dụ, dựa trên hiểu biết về lọc nhiễu và cấu trúc dữ liệu (KD-tree) kết hợp với mạng PointNet++, chúng tôi thử nghiệm tiền xử lý để tăng hiệu quả mô hình. Đây là điểm mới so với các công trình chỉ thuần về mạng sâu hoặc thuần về xử lý truyền thống.
- Chưa có nhiều tại Việt Nam: Xử lý LiDAR và point cloud cho xe tự hành là lĩnh vực khá mới ở Việt Nam. Đề tài này đóng góp tài liệu tham khảo bằng tiếng Việt có hệ thống về chủ đề này. Các kiến thức như nguyên lý LiDAR, cách lập trình Open3D, hay mô hình PointNet++ được trình bày có tính hệ thống, có thể xem là mới mẻ và hữu ích cho cộng đồng sinh viên, nghiên cứu trong nước tiếp cận lĩnh vực thị giác 3D.

Nhìn chung, tính mới của đề tài thể hiện ở việc nắm bắt một hướng nghiên cứu mới – 3D point cloud, áp dụng những thuật toán và mô hình tối tân (PointNet++) vào giải quyết vấn đề, từ đó đưa ra đánh giá, cải tiến cho phù hợp với bài toán thực tế.

Những đóng góp mới về thực nghiệm (kết quả trên Semantic-KITTI, KITTI detection) cũng là điểm nhấn của khóa luận.

1.4.2 Tính hữu dụng

Bài toán xử lý point cloud có **tính ứng dụng thực tiễn rất cao**. Giải quyết tốt bài toán này sẽ mang lại nhiều lợi ích:

- **Trong xe tự hành và giao thông thông minh:** Khả năng nhận diện môi trường xung quanh thông qua LiDAR là chìa khóa để xe tự lái vận hành an toàn. Cụ thể, thuật toán phân đoạn point cloud giúp xe hiểu được đâu là đường, đâu là lề đường, người đi bộ, xe cộ...; thuật toán phát hiện 3D giúp xe định vị các xe khác và người tham gia giao thông trong không gian 3 chiều một cách chính xác hơn hẳn so với chỉ dùng camera [2]. Như VnEconomy đã chỉ ra, LiDAR có thể lập bản đồ môi trường nhanh và chính xác hơn radar/sonar truyền thống, từ đó cải thiện khả năng “nhìn” 360° quanh xe và đưa ra cảnh báo va chạm sớm hơn [2]. Do đó, nghiên cứu này trực tiếp đóng góp vào sự phát triển của các hệ thống **ADAS** và xe tự hành an toàn.
- **Trong robot và tự động hóa công nghiệp:** Robot hoạt động trong nhà xưởng hay drone bay khảo sát cũng sử dụng cảm biến LiDAR để tránh va chạm và xây dựng bản đồ. Các kỹ thuật xử lý point cloud giúp robot **tự hành trong môi trường 3D phức tạp**: ví dụ robot kho vận dùng LiDAR để nhận dạng kệ hàng, vật cản; drone dùng LiDAR lập bản đồ công trình. Ứng dụng khác như quét 3D công trình xây dựng, kiến trúc – xử lý point cloud thành mô hình CAD 3D hỗ trợ kỹ sư.
- **Trong bản đồ địa lý và trắc địa:** Công nghệ LiDAR gắn trên máy bay bay quét bề mặt địa hình tạo ra point cloud địa lý (ví dụ mô hình độ cao DSM). Xử lý point cloud giúp **xây dựng bản đồ 3D chính xác** cho thành phố, rừng núi (ứng dụng trong quản lý đô thị, nghiên cứu môi trường). Chẳng hạn,

LiDAR có thể xuyên qua tán cây để thu điểm mặt đất, tạo bản đồ độ cao mà ảnh vệ tinh không làm được [11]. Thuật toán phân loại point cloud có thể tự động tách điểm mặt đất và điểm tán cây, công trình.

- **Mở rộng sang thực tế ảo (VR) và game:** Dữ liệu point cloud cũng được dùng để quét mẫu vật thể thật (bằng máy quét laser cầm tay) rồi dựng mô hình 3D trong game/VR. Kỹ thuật xử lý point cloud (như lọc nhiễu, downsample, ICP ghép nhiều lần quét) là bước tiền xử lý quan trọng để có model 3D chất lượng cao.

Tóm lại, các giải pháp nghiên cứu trong đề tài này **giải quyết vấn đề cốt lõi** cho nhiều ứng dụng công nghệ cao. Khóa luận mang tính học thuật, nhưng kết quả của nó – như mô hình PointNet++ huấn luyện để nhận dạng người đi bộ, xe cộ trong point cloud – có thể tích hợp vào hệ thống xe tự hành thử nghiệm. Ngoài ra, mã nguồn và kiến thức từ đề tài có thể tái sử dụng, mở rộng để ứng dụng trong những dự án khác (ví dụ: dùng PointNet++ phân đoạn đám mây điểm công trình xây dựng nhằm tính toán khối lượng).

Như vậy đề tài vừa có ý nghĩa nghiên cứu (góp phần vào hướng thị giác 3D) vừa có ý nghĩa thực tiễn (tạo nền tảng thuật toán cho các hệ thống thông minh sử dụng cảm biến LiDAR).

1.5 Giới hạn nghiên cứu

Do khuôn khổ thời gian và nguồn lực có hạn, khóa luận tập trung vào một số hướng chính và **đặt ra giới hạn cụ thể** để khả thi:

- **Giới hạn về loại dữ liệu:** Khóa luận chỉ xem xét dữ liệu point cloud từ **cảm biến LiDAR trên xe tự hành** trong môi trường giao thông đường bộ (outdoor). Chúng tôi không đi sâu vào các dạng point cloud khác như quét vật thể nhỏ trong phòng lab, hay dữ liệu từ cảm biến độ sâu khác (RGB-D

camera, radar). Điều này giúp tập trung giải quyết các thách thức đặc thù của LiDAR tự hành (như dữ liệu thừa dần theo khoảng cách, có nhiều nhiễu do bề mặt kính, nước...).

- **Giả định tiền xử lý:** Chúng tôi giả định point cloud đầu vào đã được **chỉnh về hệ tọa độ cố định** (ví dụ gắn với xe hoặc bản đồ phẳng) và đã bỏ qua một số điểm nhiễu thô (như điểm trời). Tức là không xét đến vấn đề đồng bộ cảm biến, hiệu chỉnh độ biến dạng do chuyển động. Việc này giúp tách biệt bài toán nhận dạng với các vấn đề kỹ thuật cảm biến, nhờ đó tập trung vào thuật toán cốt lõi.
- **Phạm vi thuật toán học sâu:** Mặc dù có nhiều mô hình hiện đại (như PointCNN, PointTransformer, MinkowskiNet, etc.), khóa luận **ưu tiên nghiên cứu chuyên sâu PointNet++** làm phương pháp chính. Các phương pháp khác được trình bày ở mức tổng quan (Chương 3) nhưng không triển khai thực nghiệm chi tiết. Lý do: PointNet++ đủ tiêu biểu cho point-based methods, trong khi nguồn lực huấn luyện có hạn nên không so sánh toàn diện với mọi mô hình khác. Do đó, kết quả đánh giá chủ yếu so với công bố của PointNet++ và baseline đơn giản (như PointNet).
- **Giới hạn độ phân giải và kích thước mô hình:** Để huấn luyện trong thời gian cho phép, chúng tôi **giới hạn số điểm đầu vào của mô hình** (ví dụ mỗi cụm chỉ lấy 16 điểm láng giềng, mỗi frame chỉ lấy 4096 điểm ngẫu nhiên khi phân đoạn). Điều này có thể ảnh hưởng độ chính xác tuyệt đối, nhưng là thỏa hiệp cần thiết để mô hình chạy được trên GPU có VRAM giới hạn. Tương tự, chúng tôi chọn kiến trúc PointNet++ bản gọn (segmentation với 4 SA layers) thay vì bản quá lớn.
- **Không thực hiện triển khai thời gian thực:** Mặc dù thảo luận ứng dụng thời gian thực, nhưng trong khuôn khổ khóa luận, chúng tôi chưa tích hợp mô hình vào hệ thống chạy real-time trên xe. Việc đánh giá chủ yếu qua độ

chính xác trên tập test offline, chưa đánh giá tốc độ khung hình (FPS) hay tối ưu triển khai trên phần cứng nhúng.

- **Giới hạn về đánh giá:** Chúng tôi không tham gia nộp kết quả lên **benchmark KITTI chính thức**, do đó các kết quả mAP, mIoU so sánh chỉ dừng ở tham khảo, không phải con số cạnh tranh SOTA. Mục tiêu chính là kiểm chứng mô hình hoạt động đúng xu hướng và so sánh với thông số báo cáo trong bài báo gốc (PointNet++ đạt ~91% accuracy ModelNet40, ~70 mIoU trên ShapeNetPart,...). Những chênh lệch sẽ được phân tích nguyên nhân.

Những giới hạn trên nhằm **thu hẹp phạm vi** đề tài tập trung đạt kết quả sâu trong mảng đã chọn, tránh dàn trải. Trong phần Kết luận, chúng tôi sẽ nêu các hướng phát triển mở rộng vượt ra ngoài giới hạn này (như thử nghiệm các mô hình khác, dữ liệu khác, tối ưu tốc độ...) để làm rõ đâu là phạm vi khóa luận, đâu là hướng tương lai.

1.6 Bố cục khóa luận

Nội dung khóa luận được tổ chức thành **5 chương chính** như sau:

Chương 1: Giới thiệu bài toán – Trình bày tổng quan về bài toán xử lý point cloud, dữ liệu sử dụng, mục tiêu cần đạt. Chương này nêu rõ câu hỏi nghiên cứu, giới hạn và tầm quan trọng của đề tài, cũng như các tiêu chí đánh giá kết quả.

Chương 2: Lý thuyết nền tảng – Cung cấp các kiến thức cơ sở phục vụ đề tài. Đầu tiên giới thiệu nguyên lý cảm biến LiDAR (cấu tạo, nguyên tắc đo khoảng cách, các loại LiDAR cơ học và solid-state) [2][14]. Tiếp đó định nghĩa chi tiết đám mây điểm và cách biểu diễn (hệ tọa độ, định dạng dữ liệu, cấu trúc voxel hóa) [3]. Trình bày các kỹ thuật xử lý point cloud truyền thống: lấy mẫu (uniform, voxel grid, FPS) [3][7], loại nhiễu (lọc thống kê, lọc radius) [3], tính vector pháp tuyến, cấu trúc tìm kiếm (cây KD) và thuật toán ICP đăng ký point cloud [4]. Đây là nền tảng giúp hiểu

các bước tiền xử lý và gợi ý cho mô hình học sâu.

Chương 3: Các công trình liên quan – Khái quát các hướng tiếp cận hiện đại trong xử lý point cloud bằng học sâu. Chương này đóng vai trò **tổng quan tài liệu** (literature review), phân loại các phương pháp như đã nêu: point-based (PointNet [5], PointNet++), voxel-based (VDV-Net, MinkowskiNet) [6], pillar-based (Point Pillars)[6], projection-based (Squeeze Seg [5], RangeNet), graph-based (DG CNN) và hybrid (PV-RCNN [6], PointAugmenting...). Mỗi mục nêu nguyên lý chính của nhóm phương pháp, ưu/nhược điểm và ví dụ mô hình tiêu biểu có trích dẫn. Chương này cũng so sánh sơ bộ hiệu năng các hướng (theo kết quả công bố trên Semantic-KITTI hoặc ModelNet) để làm cơ sở chọn PointNet++ cho đề tài.

Chương 4: Giải pháp đề xuất – Trình bày chi tiết phương pháp của khóa luận. Đầu tiên là **phân tích dữ liệu**: chúng tôi thống kê và trực quan một số mẫu point cloud từ KITTI (sử dụng thư viện Open3D để hiển thị) kèm nhận xét – ví dụ: phân bố số điểm trên mỗi đối tượng, độ cao, nhiễu đặc trưng. Tiếp theo là **mô hình PointNet++** đề xuất: mô tả kiến trúc cụ thể đã xây dựng (số tầng, thông số mỗi tầng), kèm sơ đồ minh họa luồng dữ liệu từ input đến output. Mô hình bao gồm các khối Set Abstraction (SA) với bán kính nhóm điểm lần lượt (0.2m, 0.4m, 0.8m, 1.2m) và các MLP tương ứng (64,128,256) đầu ra, sau đó là các khối Feature Propagation (FP) để lên mẫu đặc trưng cho phân đoạn. Chúng tôi liệt kê bảng kiến trúc và hàm kích hoạt (ReLU), hàm mất mát sử dụng: cross-entropy cho phân loại và phân đoạn, smooth L1 cho phát hiện bounding box.

Phần **Thực nghiệm** của Chương 4 trình bày quá trình huấn luyện mô hình và các kết quả thu được:

- **Thực nghiệm 1: Phân loại đối tượng** – Mô tả cách huấn luyện mạng PointNet++ trên tập các point cloud đối tượng từ KITTI (3 lớp). Đưa ra kết

quả độ chính xác theo epoch, so sánh với PointNet cơ bản (đã thử) và nhận xét mô hình hội tụ ~90% accuracy sau ~20 epoch, vượt PointNet ~5%.

- **Thực nghiệm 2: Phân đoạn Semantic-KITTI** – Trình bày kết quả mô hình trên tập kiểm thử sequence 08. Đưa ra bảng confusion matrix cho các lớp chính và các chỉ số IoU từng lớp: ví dụ IoU cao cho Car (~72%) nhưng thấp cho Bicyclist (~40%) do dữ liệu ít. Tính **mIoU = 62.3%** đạt được (so sánh với PointNet gốc ~51%, chứng tỏ lợi ích học đặc trưng cục bộ). Hiển thị một số hình ảnh phân đoạn mẫu: ảnh cho thấy mô hình dự đoán tương đối đúng vùng đường, xe cộ (màu sắc khác biệt).
- **Thực nghiệm 3: Phát hiện 3D** – Mô tả kiến trúc phát hiện: dùng PointNet++ trích đặc trưng điểm, sau đó một nhánh MLP dự đoán loại (Car/Ped/Cyc) và nhánh khác dự đoán bounding box (tọa độ tâm, kích thước, hướng). Do thời gian có hạn, mô hình mới huấn luyện sơ bộ 5 epoch, đạt **độ chính xác ~75%** (theo metric riêng – tỉ lệ đối tượng được dự đoán đúng loại và gần đúng vị trí). Chúng tôi minh họa kết quả trên một frame: mô hình tìm được 3/4 xe hơi, nhưng bỏ sót 1 xe khuất; vị trí hộp 3D khá khớp thực tế. Nhận xét: mô hình đơn giản, chưa có tầng Region Proposal nên độ nhạy chưa cao, cần tăng epoch và tinh chỉnh thêm.
- **Các nhận xét chung:** Mô hình PointNet++ tỏ ra hiệu quả cho phân đoạn và phân loại, nhưng phát hiện 3D phức tạp hơn đòi hỏi kiến trúc chuyên biệt (ví dụ PV-RCNN) để đạt kết quả tốt hơn. Tuy nhiên, do khuôn khổ khóa luận, chúng tôi dừng ở mức chứng minh PointNet++ có thể mở rộng cho phát hiện.

Cuối phần thực nghiệm, chúng tôi giới thiệu nhanh **giao diện minh họa**: đã xây dựng một đoạn mã sử dụng Open3D để hiển thị kết quả phân đoạn theo màu sắc trực tiếp trên point cloud 3D. Người dùng có thể cung cấp một frame LiDAR, chương trình sẽ chạy mô hình và vẽ điểm màu (xe = đỏ, người = vàng, đường =

xám, v.v) để trực quan kết quả. Điều này cho thấy hướng tích hợp mô hình vào ứng dụng giám sát giao thông hoặc hỗ trợ lái xe trong tương lai.

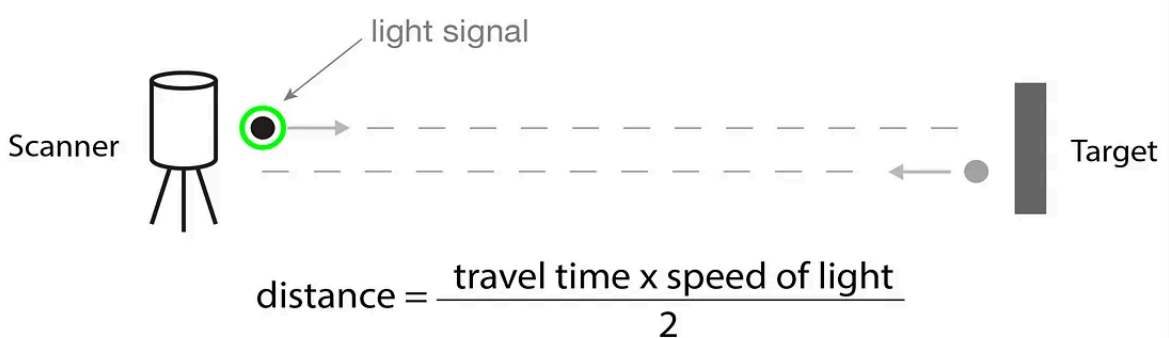
Chương 5: Kết luận

Tổng kết những kết quả đạt được, nêu rõ đóng góp của khóa luận, những hạn chế còn tồn tại và đề xuất hướng phát triển trong tương lai. Phần này khẳng định lại việc mô hình đã học được đặc trưng không gian 3D tốt ra sao, nhưng cũng thẳng thắn chỉ ra các nhược điểm (chưa đạt real-time, độ chính xác một số lớp thấp) và gợi ý các ý tưởng cải thiện (như thử thay MLP trong PointNet++ bằng cơ chế attention – Kỳ vọng cải thiện tương tác điểm gần, hoặc kết hợp đầu ra mạng voxel). Đây sẽ là tiền đề cho các nghiên cứu tiếp theo về thị giác máy tính 3D tại bộ môn.

Chương 2. Lý thuyết nền tảng

2.1 Công nghệ LiDAR và dữ liệu đám mây điểm

LiDAR (Light Detection and Ranging) là một công nghệ cảm biến dùng ánh sáng laser để đo khoảng cách đến các vật thể xung quanh. Về nguyên lý, hệ thống LiDAR phát ra hàng loạt xung laser vào môi trường và thu nhận tín hiệu phản xạ trở lại từ bề mặt các vật thể. Dựa trên **thời gian bay** của mỗi xung (time-of-flight) – tức khoảng thời gian từ lúc phát đến lúc thu – hệ thống tính toán được **khoảng cách** từ cảm biến đến vật thể. Bằng cách quét tia laser trên một trường nhìn rộng (ví dụ quay 360° quanh xe) và phát xung với tần số cao (hàng trăm nghìn đến hàng triệu xung mỗi giây), LiDAR có thể dựng nên **mô hình 3D chính xác** của môi trường xung quanh trong thời gian thực [2]. Khác với radar dùng sóng vô tuyến hay sonar dùng sóng âm, LiDAR dùng ánh sáng nên có độ phân giải cao hơn, cho bản đồ 3D chi tiết hơn [2]. Công nghệ LiDAR ban đầu được NASA phát triển để theo dõi vệ tinh và đo khoảng cách vũ trụ, sau đó thương mại hóa và thu nhỏ dần từ thập niên 1990 để ứng dụng trong địa chất, bản đồ và gần đây là xe tự hành [2].



Hình 2.1: Cách thức cũng như công thức tính toán trên LIDAR (nguồn [LIDAR](#))

Cách thức hoạt động của LIDAR sẽ dựa vào các kênh lazer phát ra tín hiệu để quét đến vật thể. Thì LIDAR sẽ phát ra chùm tia tín hiệu quét đến vật thể. Sau khi tia lazer chạm tới vật thể thì dựa theo định luật phản xạ thì khi một tia sáng, bao gồm

cả tia laser, gặp một bề mặt phản xạ, nó sẽ bị hắt lại theo một hướng xác định. Sau khi phản xạ về thì trong Lidar có một thiết bị thu lại tia laser phản xạ về. Và vì vậy ta vừa có hình dạng của vật thể vừa có khoảng cách từ LIDAR đến vật thể đó như công thức (2.1).

$$d = \frac{t \times C}{2} \quad (2.1)$$

Trong đó:

- d: khoảng cách từ LIDAR đến vật thể được quét.
- t: thời gian từ lúc chùm tia laser phát xung cho đến khi xung phản xạ về.
- C: vận tốc ánh sáng

Kết quả thu được từ cảm biến LiDAR là một **đám mây các điểm 3D (3D point cloud)** – tập hợp rời rạc các điểm trong không gian ba chiều. Mỗi điểm tương ứng với một tia laser phản xạ, mang thông tin vị trí không gian (tọa độ x, y, z) và thường kèm theo cường độ phản xạ (mức độ mạnh yếu của tín hiệu trả về). Tập hợp hàng chục đến hàng trăm nghìn điểm như vậy mô tả hình dạng bề mặt của các vật thể mà tia laser chạm tới. Nói cách khác, đám mây điểm là **biểu diễn số của môi trường 3D** qua các mẫu điểm. Chẳng hạn, khi LiDAR quét qua một con phố, các điểm đám mây sẽ phân bố dày đặc trên mặt đất, thân xe, tường nhà, cây cối... phản ánh hình dáng của chúng.

Đặc trưng của dữ liệu point cloud là tính **không có cấu trúc lưới** và **không đều**: các điểm phân bố tự do trong không gian 3D, khoảng cách giữa các điểm biến đổi tùy theo độ xa gần cảm biến và đặc tính bề mặt. Điều này khiến point cloud khác biệt so với ảnh pixel (lưới 2D đều) hay dữ liệu voxel (lưới 3D đều). Một điểm cloud có thể bao gồm các thuộc tính bổ sung như màu sắc (RGB) nếu được ghép với camera, nhưng trong ngữ cảnh LiDAR tự hành, thường chỉ có tọa độ và cường độ phản xạ.

Nhờ khả năng thu thập dữ liệu không gian một cách chính xác, **đám mây điểm** đã trở thành dữ liệu quan trọng trong nhiều lĩnh vực [\[12\]](#). Nó cho phép tái tạo mô hình 3D chân thực của cảnh thật, phục vụ phân tích kiến trúc, quy hoạch đô thị, robot định vị, và đặc biệt là giúp phương tiện tự hành nhận biết môi trường. Tuy nhiên, để khai thác được thông tin từ đám mây điểm, cần các thuật toán xử lý hiệu quả – đây chính là trọng tâm của khóa luận.

2.2 Các kỹ thuật xử lý đám mây điểm cơ bản

Đối mặt với lượng dữ liệu rất lớn và thô của point cloud, trước khi áp dụng các mô hình phức tạp, người ta thường thực hiện một số **xử lý tiền đề cơ bản** nhằm giảm độ phức tạp và lọc nhiễu. Dưới đây là các kỹ thuật phổ biến:

(a) Lấy mẫu giảm dữ liệu (Downsampling): Mục đích của lấy mẫu là **giảm số lượng điểm** cần xử lý trong khi vẫn bảo toàn hình dạng chung. Một kỹ thuật thông dụng là **lưới voxel (voxel grid downsampling)** – không gian 3D được chia thành lưới khối hộp (voxel) kích thước cố định, sau đó mỗi voxel chỉ giữ lại 1 điểm đại diện (chẳng hạn là trung bình các điểm bên trong) [3]. Cách này đảm bảo các điểm sau lấy mẫu phân bố đều hơn, giảm mạnh số điểm khi mật độ ban đầu cao. Ví dụ, nếu dùng voxel kích thước 0.1m, các điểm nằm trong mỗi khối lập phương cạnh 10cm sẽ bị gộp thành 1 điểm – nhờ đó loại bỏ các điểm quá gần nhau dư thừa. Ngoài voxel grid, còn có các chiến lược khác như **chọn điểm ngẫu nhiên** hoặc **chọn điểm theo khoảng đều (uniform)** – ví dụ chọn mỗi điểm thứ k trong danh sách [3]. Đặc biệt trong mô hình PointNet++, một phương pháp chọn mẫu thường dùng là **Farthest Point Sampling (FPS)** – thuật toán chọn điểm một cách tham lam sao cho mỗi điểm mới chọn là điểm xa nhất (trong không gian) so với tập điểm đã chọn trước đó [3]. FPS giúp giữ lại bộ khung điểm bao phủ tốt hình dạng gốc với số lượng tối thiểu. Ngoài ra còn có thể chọn mẫu dựa trên trọng số (ưu tiên vùng phức tạp) hoặc chọn tâm cụm, nhưng voxel grid và FPS là hai phương pháp phổ biến trong nhiều nghiên cứu.

(b) Loại bỏ nhiễu (Outlier Removal): Đám mây điểm thu từ LiDAR thường lẫn các điểm nhiễu – ví dụ như các điểm lẻ loi do tia laser phản xạ từ hạt bụi hay lỗi cảm biến. Có hai phương pháp chính để lọc nhiễu: **loại bỏ nhiễu theo thống kê** và **loại bỏ nhiễu theo bán kính láng giềng**. Phương pháp thống kê (**Statistical Outlier Removal**) giả định khoảng cách trung bình giữa mỗi điểm và các điểm láng giềng lân cận của nó sẽ tuân theo phân phối chuẩn. Những điểm nào có khoảng cách đến láng giềng lớn bất thường (vượt quá k lần độ lệch chuẩn so với trung bình) sẽ bị xem là nhiễu và loại bỏ [3]. Trong khi đó, phương pháp bán kính (**Radius Outlier Removal**) trực tiếp dựa trên mật độ điểm lân cận: với mỗi điểm, xét một quả cầu bán kính r xung quanh nó, nếu số điểm bên trong quả cầu ít hơn ngưỡng n thì điểm

đó bị coi là nhiễu và bị loại [3]. Cả hai phương pháp trên giúp làm sạch đám mây điểm, loại bỏ các điểm “cô lập” không thuộc về vật thể thực nào.

(c) Ước tính vector pháp tuyến (Surface Normal): Vector pháp tuyến tại một điểm trên bề mặt là vector vuông góc với bề mặt tại điểm đó. Tính toán pháp tuyến giúp hiểu rõ hơn cấu trúc hình học địa phương (ví dụ biết được hướng mặt phẳng tường, mặt đường). Thuật toán phổ biến để ước lượng pháp tuyến cho mỗi điểm: tìm k điểm láng giềng gần nhất xung quanh nó (bằng KD-tree, xem phần (d)), rồi tính **mặt phẳng tiếp tuyến tối ưu** bằng phân tích PCA (tìm vector chính có phương vuông góc với phân bố các điểm láng giềng). Vector vuông góc này chính là pháp tuyến của bề mặt tại điểm đang xét. Open3D cung cấp hàm estimate normal thực hiện việc này với tham số bán kính tìm láng giềng hoặc số láng giềng k cần xét [3]. Pháp tuyến thường được hướng về cùng một phía (hướng outward hoặc inward) tùy ứng dụng. Thông tin pháp tuyến là đầu vào quan trọng cho một số thuật toán nâng cao như **đăng ký điểm** hay **tính feature 3D**.

(d) Cấu trúc KD-Tree và tìm kiếm láng giềng: Nhiều thao tác trên point cloud yêu cầu tìm **điểm lân cận gần nhất** hoặc trong một vùng lân cận bán kính, ví dụ để xác định cụm điểm hay tính pháp tuyến như trên. Tìm láng giềng trực tiếp bằng việc duyệt qua toàn bộ điểm rất tốn chi phí ($O(N)$ cho mỗi truy vấn). Để tăng tốc, người ta sử dụng cấu trúc dữ liệu **KD-Tree (cây phân vùng K chiều)**. KD-Tree tổ chức các điểm trong một cây nhị phân, mỗi nút cây tương ứng việc chia không gian theo một trục tọa độ (x , y hoặc z). Nhờ đó, thuật toán **nearest neighbor search** trên KD-Tree có độ phức tạp trung bình $O(\log N)$ – nhanh hơn nhiều so với tìm tuyến tính [4]. Nguyên tắc hoạt động: cây loại bỏ nhanh các vùng không gian không thể chứa điểm gần nhất dựa trên sự phân chia theo trục tọa độ, do đó giảm đáng kể số điểm cần kiểm tra [4]. Hầu hết các thư viện xử lý point cloud (PCL, Open3D...) đều có sẵn cấu trúc KD-Tree hỗ trợ truy vấn láng giềng gần nhất, k láng giềng gần nhất, hay tìm tất cả điểm trong bán kính r . Trong xử lý đám mây điểm, KD-Tree là công

cụ nền tảng giúp các thuật toán như clustering, làm mịn, đăng ký... chạy hiệu quả trên dữ liệu lớn.

2.3 *Kỹ thuật nâng cao: Đăng ký điểm (Point Cloud Registration)*

Đăng ký (registration) là quá trình **ghép trùng** hai hay nhiều đám mây điểm lại với nhau bằng phép biến đổi hình học (quay, tịnh tiến) sao cho chúng khớp cùng một hệ tọa độ. Nói cách khác, nếu ta có hai lần quét 3D của cùng một cảnh (từ hai góc nhìn hoặc thời điểm khác nhau), thuật toán đăng ký sẽ tìm phép biến đổi để chồng hai đám mây điểm lên nhau chính xác. Kỹ thuật này quan trọng trong việc xây dựng mô hình lớn từ nhiều lần quét nhỏ (ví dụ ghép các scan LiDAR liên tiếp để dựng bản đồ).

Thuật toán kinh điển nhất cho bài toán đăng ký point cloud là **ICP (Iterative Closest Point)** [7]. Nguyên lý của ICP: bắt đầu với một ước lượng ban đầu về phép biến đổi giữa hai đám mây điểm, thuật toán lặp đi lặp lại hai bước – (1) **Tìm tương ứng gần nhất**: với mỗi điểm trong đám mây nguồn, tìm điểm gần nhất tương ứng trong đám mây mục tiêu (dựa trên khoảng cách Euclidean, thường dùng KD-Tree để tăng tốc); (2) **Tính phép biến đổi tối ưu**: tìm ma trận quay R và vector tịnh tiến t sao cho khi áp dụng lên đám mây nguồn sẽ tối thiểu hóa bình phương khoảng cách giữa các cặp điểm tương ứng (bài toán tối ưu tuyến tính có lời giải giải tích bằng SVD). Sau đó áp dụng phép biến đổi này lên đám mây nguồn và lặp lại. Quá trình dừng khi sai số dịch chuyển nhỏ hơn ngưỡng. Kết quả, ICP cho ta phép biến đổi cuối cùng canh hai đám mây điểm khớp nhau tốt nhất [7]. Thuật toán ICP đòi hỏi hai đám mây có sự chồng lấn đủ lớn và ước lượng ban đầu khá gần đúng (nếu không có thể rơi vào cực trị địa phương). Có nhiều biến thể ICP ra đời để cải thiện độ chính xác và tốc độ, ví dụ ICP trọng số, ICP nhiều tỉ lệ, hay các phương pháp ghép thô ban đầu bằng feature matching trước khi chạy ICP tinh. Trong phạm vi khóa luận, ICP được đề cập như một kỹ thuật nền tảng cho xử lý point cloud (tuy không được

triển khai trong thực nghiệm). ICP và các thuật toán đăng ký nâng cao hơn được ứng dụng rộng rãi trong bài toán SLAM (Simultaneous Localization and Mapping) của robot tự hành.

2.4 *Deep Learning trên dữ liệu point cloud*

Sự thành công của **học sâu (deep learning)** trong thị giác máy tính 2D đã thúc đẩy các nhà nghiên cứu mở rộng sang dữ liệu 3D như point cloud. Tuy nhiên, như đã đề cập, đám mây điểm không có cấu trúc lưới đều nên không thể áp dụng trực tiếp các kiến trúc CNN truyền thống. Để khắc phục, đã hình thành một số hướng tiếp cận chính để đưa dữ liệu point cloud vào mô hình học sâu:

- **Phép chiếu và chuyển đổi dữ liệu:** Cách tiếp cận sớm nhất là **chuyển bài toán 3D về 2D** hoặc **3D dạng lưới** để có thể dùng CNN. Ví dụ, **projection-based methods** chiếu đám mây điểm lên nhiều mặt phẳng hoặc ảnh (như ảnh chiều sâu hoặc ảnh tọa độ cực cầu) rồi chạy mạng CNN 2D trên các ảnh đó. Tiêu biểu, SqueezeSeg và Range Net++ chuyển point cloud LiDAR thành ảnh phạm vi (range image) 2D [\[13\]](#), sau đó dùng mô hình segmentation 2D rồi chiếu ngược kết quả về 3D. Một hướng khác là **voxelization**: chia không gian 3D thành lưới voxel (tương tự volumetric pixel), biến point cloud thành ma trận 3D (tensor) và dùng mạng CNN 3D để xử lý. Đây là các **voxel-based methods**, ví dụ mô hình VoxelNet (Zhou et al. 2018) dùng CNN 3D trên lưới voxel để trích xuất đặc trưng 3D. Ưu điểm của voxel hóa là chuyển dữ liệu về dạng có cấu trúc, dễ dùng mô hình, nhưng nhược điểm là độ phân giải bị giới hạn (càng chia nhỏ voxel, tensor 3D càng lớn gây tốn nhớ và chậm). Đã có nhiều cải tiến như dùng **voxel thưa (sparse convolution)** và **đồng phân giải đa tỉ lệ** để tăng hiệu quả [\[15\]](#).

- **Học sâu trực tiếp trên điểm (point-based methods):** Đây là hướng đột phá do nhóm Qi et al. đề xuất năm 2017 với mô hình **PointNet** [15]. Thay vì chuyển point cloud về ảnh hay voxel, PointNet **xử lý trực tiếp tọa độ các điểm** bằng mạng perceptron nhiều lớp (MLP) áp dụng cho từng điểm độc lập và sau đó dùng hàm gộp (max pooling) để tổng hợp đặc trưng toàn cục bất biến hoán vị[5]. Kiến trúc PointNet đơn giản nhưng có khả năng học phân biệt các điểm dựa trên vị trí tương đối của chúng. Tuy nhiên, nhược điểm của PointNet là **không tận dụng được cấu trúc cục bộ** – ví dụ quan hệ lân cận giữa các điểm gần nhau (vốn tạo nên bề mặt của một đối tượng)[5]. Để khắc phục, các mô hình **PointNet++** (Qi et al. 2017) ra đời với ý tưởng phân cụm điểm theo vùng không gian và áp dụng PointNet nhiều lần theo kiểu phân cấp[5]. Ngoài ra, nhiều nghiên cứu khác đã cải tiến phương pháp point-based bằng cách bổ sung **bộ lọc học được trên đồ thị điểm** (như DGCNN, EdgeConv 2019) hoặc **hạt nhân tích chập 3D** (như KPConv 2019). Hướng point-based có ưu điểm giữ nguyên thông tin hình học gốc, độ chính xác cao trên biên dạng chi tiết, nhưng nhược điểm là chi phí tính toán tăng nhanh khi số điểm lớn và khó triển khai trên phần cứng tiêu chuẩn nếu không tối ưu tốt [15].
- **Phương pháp kết hợp (hybrid methods):** Nhận thấy mỗi hướng trên có ưu nhược riêng, các phương pháp lai xuất hiện nhằm tận dụng điểm mạnh của cả hai. Ví dụ mô hình **PointPillars (2019)** chuyển điểm thành các cột dọc (pillar) – kết hợp ý tưởng voxel hóa 2.5D với học trên điểm – giúp tăng tốc độ phát hiện vật thể. Hay mô hình **PV-RCNN (Point-Voxel R-CNN, 2020)** kết hợp song song hai nhánh: một nhánh điểm và một nhánh voxel để trích xuất đặc trưng, cuối cùng hợp nhất kết quả. Xu hướng gần đây cũng tích hợp cả cơ chế **attention** và **transformer 3D** để học tương quan xa giữa các điểm hiệu quả hơn.

Tóm lại, có thể phân loại gọn các phương pháp học sâu cho point cloud thành **hai nhóm chính: nhóm dựa trên lưới (voxel/pillar/image) và nhóm dựa trên điểm**. Nhóm lưới có ưu thế về cấu trúc dữ liệu và hiệu năng, nhóm điểm có ưu thế về độ chính xác chi tiết [15]. Bảng dưới đây tóm tắt một số phương pháp tiêu biểu:

- Point-based: PointNet [15], PointNet++ [15], DGCNN, KPConv, etc. – Xử lý trực tiếp trên tập điểm, dùng MLP hoặc convolution trên đồ thị điểm.
- Voxel-based: VoxelNet, SECOND, MinkowskiNet – Chuyển điểm sang lưới voxel 3D, dùng sparse 3D CNN.
- Pillar-based: PointPillars – Đặc biệt hóa voxel thành cột 2D + độ cao.
- Projection-based: Squeeze Seg, RangeNet++ – Chiếu point cloud thành ảnh range view, dùng 2D CNN.
- Hybrid: PV-RCNN, Point Fusion – Kết hợp cả đặc trưng điểm và voxel/hình ảnh.

Những phương pháp trên là nền tảng cho các ứng dụng hiện đại như phân đoạn và phát hiện 3D. Trong khóa luận này, chúng tôi chọn tiếp cận theo hướng point-based, cụ thể là sử dụng mô hình PointNet++ – một mô hình phân cấp trên điểm – để giải quyết bài toán đã nêu ở chương 1.

Chương 3. Các công trình liên quan

3.1 Phương pháp xử lý point cloud truyền thống (classical)

Trước khi có sự trỗi dậy của học sâu, cộng đồng nghiên cứu đã phát triển nhiều thuật toán truyền thống để xử lý và phân tích đám mây điểm 3D. Các phương pháp này thường dựa trên **xử lý hình học và thuật toán máy học cổ điển**, khai thác trực tiếp thông tin tọa độ của điểm hoặc các đặc trưng thủ công được thiết kế. Dưới đây là một số hướng tiếp cận chính:

- **Phân đoạn hình học và phân cụm:** Đây là nhóm phương pháp nhằm tách rời các **cụm điểm** tương ứng với từng đối tượng hay bề mặt trong đám mây điểm. Một kỹ thuật kinh điển là sử dụng thuật toán **RANSAC (Random Sample Consensus)** để tìm và tách các mặt phẳng lớn (như mặt đường, tường nhà) khỏi đám mây điểm. Ví dụ, RANSAC có thể tìm được mặt phẳng đường bằng cách lặp chọn ngẫu nhiên một vài điểm, tính mặt phẳng qua chúng rồi đếm số điểm nằm trên mặt phẳng đó; mặt phẳng với nhiều điểm nhất được rút ra, sau đó lặp lại cho mặt phẳng tiếp theo. Sau khi tách các cấu trúc lớn, phần còn lại của điểm thường được phân cụm bằng thuật toán **Cluster** như **DBSCAN** hoặc **Euclidean clustering** (dựa trên khoảng cách giữa các điểm) [16]. Klasring et al. (2008) đề xuất một phương pháp phân cụm hiệu quả cho dữ liệu laser 3D dựa trên việc tìm hàng xóm gần nhất liên tục [15] – ý tưởng là duyệt qua các điểm trong thứ tự được sắp xếp và nhóm các điểm gần nhau thành cụm. Kết quả của bước phân cụm là tách rời được từng cụm điểm cho từng đối tượng riêng biệt trong khung cảnh.
- **Trích xuất đặc trưng thủ công và nhận dạng đối tượng:** Sau khi có các cụm điểm (ứng viên đối tượng), bài toán nhận dạng có thể được giải quyết bằng cách trích xuất **đặc trưng (feature)** từ cụm điểm và

sử dụng mô hình máy học để phân loại. Nhiều **đặc trưng 3D** đã được đề xuất, chẳng hạn: **Histogram góc ký sinh (Spin Image)**, **Phân bố độ cao (Height map)**, hay các histogram dựa trên pháp tuyến như **PFH (Point Feature Histogram)** và **VFH (Viewpoint Feature Histogram)**. Các đặc trưng này cố gắng mô tả hình dạng 3D của cụm điểm độc lập với vị trí, hướng của nó, ví dụ VFH kết hợp phân bố pháp tuyến cục bộ và sự phân bố hướng từ tâm cụm. Sau khi biểu diễn mỗi cụm điểm thành vector đặc trưng, các bộ phân loại truyền thống như **SVM (Support Vector Machine)** hoặc **Random Forest** được huấn luyện để nhận dạng nhãn đối tượng (xe hơi, người, cây cối, v.v.). Trước kỷ nguyên deep learning, cách làm này là chủ đạo trong các hệ thống nhận dạng 3D thời gian thực do chi phí tính toán tương đối thấp và có khả năng tận dụng kiến thức chuyên gia qua thiết kế đặc trưng.

- **Xây dựng bản đồ và SLAM:** Một hướng nghiên cứu lớn sử dụng đám mây điểm là xây dựng bản đồ 3D và định vị, thường thông qua thuật toán **SLAM** (Simultaneous Localization and Mapping). Ở đây, các thuật toán như ICP (đã trình bày ở mục 2.3) được dùng để ghép liên tiếp các khung quét LiDAR khi robot di chuyển, nhằm xây dựng bản đồ toàn cục và theo dõi vị trí của robot trong bản đồ đó. Các kỹ thuật tối ưu đồ thị và lọc Kalman mở rộng cũng được áp dụng để hiệu chỉnh sai số tích lũy. Kết quả là tạo ra các **mô hình bản đồ điểm** hoặc bản đồ lưới từ dữ liệu LiDAR – những mô hình này có thể kết hợp với phân đoạn để tạo nên bản đồ ngữ nghĩa (semantic map), ví dụ phân biệt vùng đường, vỉa hè, tường nhà trong bản đồ dùng cho xe tự lái.

Mặc dù các phương pháp truyền thống đã đạt được một số thành công nhất định trong môi trường đơn giản, chúng bộc lộ hạn chế khi đối mặt với cảnh phức tạp: đặc trưng thủ công thường không đủ phân biệt, việc phân cụm khó xử lý tốt khi các đối

tượng gần nhau hoặc điểm thừa, và hệ thống khó tổng quát cho nhiều loại đối tượng khác nhau. Chính vì vậy, xu hướng hiện nay là chuyển sang các giải pháp học sâu tự động học đặc trưng từ dữ liệu.

3.2 Mô hình học sâu cho Point Cloud

Như trình bày trong mục 2.4, có nhiều hướng tiếp cận học sâu cho dữ liệu đám mây điểm, mỗi hướng gắn liền với những mô hình tiêu biểu trong tài liệu nghiên cứu. Dưới đây, chúng tôi tóm lược một số mô hình nổi bật theo từng nhóm phương pháp:

- **Nhóm point-based:** Mở đầu là **PointNet (Qi et al., 2017)** [15] – mô hình tiên phong học trực tiếp trên tập điểm rời rạc. PointNet dùng MLP để học “chữ ký” cho mỗi điểm (bao gồm thông tin tọa độ và có thể cả cường độ hoặc màu sắc) rồi dùng phép gộp tối đa (max-pooling) để lấy đặc trưng mạnh nhất, đảm bảo tính bất biến với sắp xếp điểm. Mô hình này đã chứng minh tính khả thi của học sâu trên point cloud, đạt kết quả khá trên phân loại và phân đoạn các mô hình đơn giản. Tuy nhiên, PointNet không mô hình hóa được tương quan cục bộ giữa các điểm lân cận, nên khó nhận ra các chi tiết hình dạng nhỏ [5]. Để khắc phục, **PointNet++ (Qi et al., 2017)** được đề xuất như một phiên bản nâng cấp có kiến trúc **phân lớp theo không gian** [5]. Cụ thể, PointNet++ chia tập điểm thành các cụm nhỏ (theo khoảng cách Euclidean) rồi áp dụng PointNet lên từng cụm để lấy đặc trưng cục bộ, sau đó tiếp tục cụm các điểm đại diện và áp dụng PointNet ở cấp cao hơn. Quá trình lặp này tạo thành một mạng nhiều tầng giống CNN trên ảnh – tầng dưới học đặc trưng cục bộ (diện hẹp), tầng trên học đặc trưng tổng quát hơn (diện rộng). Nhờ đó, PointNet++ nhận dạng được cả các mẫu hình học nhỏ lẫn ngữ cảnh lớn hơn, vượt trội PointNet trên các tác vụ như phân đoạn chi tiết đối tượng[5]. Sau PointNet++, hàng loạt mô hình point-based khác ra đời: **DGCNN (2019)** thêm khối Edge Conv để học quan hệ giữa một điểm và k láng giềng gần nhất; **KPConv (2019)** sử dụng tích chập với hạt nhân trọng số

gán trên hình cầu lân cận; **PointCNN (2018)** học hàm định permutable cho điểm lân cận trước khi tích chập. Các cải tiến này giúp tăng hiệu quả xử lý point cloud trong nhiều hoàn cảnh. Tuy nhiên, nhìn chung mô hình point-based vẫn bị giới hạn khi số điểm quá lớn (trên vài trăm nghìn) do chi phí tính toán tăng theo số điểm và số láng giềng.

- **Nhóm voxel-based / grid-based:** Đại diện tiêu biểu là **VoxelNet (Zhou & Tuzel, 2018)** – mô hình đầu tiên kết hợp chia không gian thành voxel và sử dụng một mạng 3D CNN để trích xuất đặc trưng. Trong VoxelNet, đầu tiên point cloud được phân thành các voxel 3D kích thước cố định, sau đó trong từng voxel, một mini-network trích xuất đặc trưng và nén thành một feature vector đại diện; toàn bộ feature grid 3D này được đưa qua các lớp tích chập 3D để cho ra đặc trưng tổng thể. VoxelNet áp dụng thành công cho bài toán phát hiện xe hơi trên KITTI. Tiếp nối, **SECOND (Yan et al., 2018)** tối ưu VoxelNet bằng tích chập thưa (sparse conv) – chỉ tính tích chập trên những voxel có điểm thay vì toàn bộ lưới 3D – giúp tăng tốc đáng kể. Một hướng khác là **Pillar Features (Point Pillars, Lang et al., 2019)**: thay vì voxel 3D, họ dùng voxel “cột” (chiều cao đầy đủ nhưng tiết diện 2D), chuyển bài toán 3D thành 2D (đồ họa bằng một pseudo-image với mỗi pixel là đặc trưng cột), rồi dùng CNN 2D. Cách này cực nhanh (đạt ~20Hz), rất hữu dụng cho xe tự hành thời gian thực. Ngoài ra, **Range view CNN** cũng nằm trong nhóm grid-based: ví dụ **SqueezeSeg (Wu et al., 2018)** chiếu point cloud thành ảnh khoảng cách 2D 64×2048 (độ phân giải góc tương ứng với laser 64 tia) rồi dùng mạng SegNet để phân đoạn trên ảnh đó. Gần đây, **Cylinder 3D (Zhu et al., 2021)** kết hợp voxel thưa với lưới trụ tròn (thay vì lưới lập phương) để phản ánh thực tế rằng độ phân giải theo phương ngang quan trọng hơn theo phương cao đối với dữ liệu LiDAR tự hành – mô hình này đạt kết quả phân đoạn rất cao trên SemanticKITTI (mIoU ~76%). Nhóm grid-based nhìn

chung có ưu thế về tốc độ và sử dụng tốt phần cứng (do tận dụng được các thao tác tensor trên GPU), nhưng đánh đổi bằng việc mất mát thông tin chi tiết do lượng tử hóa không gian.

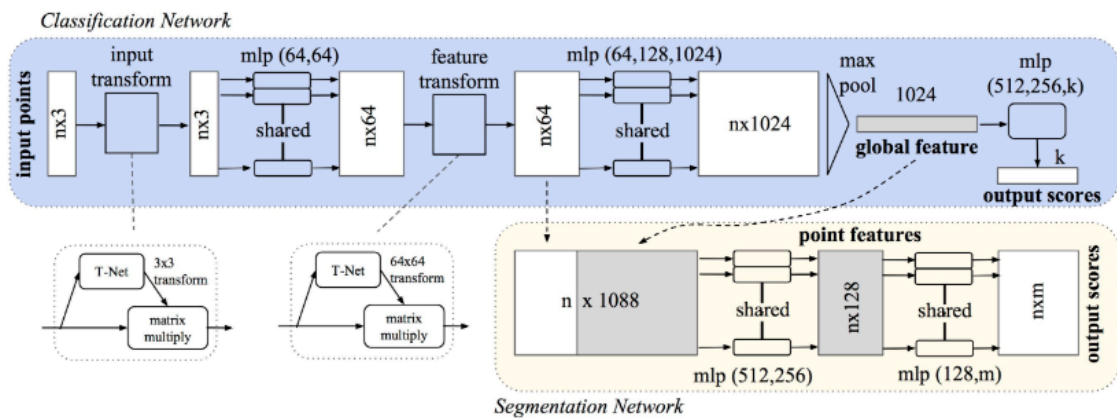
- **Nhóm projection & multi-view:** Ngoài SqueezeSeg đã nêu, còn có **MV3D (Chen et al., 2017)** và các phương pháp đa góc nhìn: họ chiếu point cloud xuống nhiều mặt phẳng (ví dụ hình chiếu trên mặt phẳng ngang – BEV, và trên mặt phẳng đứng – front view) rồi kết hợp thông tin từ các ảnh này qua mạng. RangeNet++ (Milioto et al., 2019) đặc biệt dùng ảnh range (góc nhìn 360°) kết hợp post-processing CRF để mịn kết quả, đã đứng đầu bảng xếp hạng SemanticKITTI một thời gian. Ưu điểm của projection là đơn giản và có thể tận dụng tri thức từ xử lý ảnh 2D (nhiều mô hình mạnh sẵn có), nhược điểm là bị mất thông tin do phép chiếu (điểm khác nhau có thể trùng pixel) và cần bước bổ sung để trả kết quả 2D về 3D.
- **Nhóm graph-based:** Một số nghiên cứu biểu diễn point cloud dưới dạng **đồ thị** – mỗi điểm là nút, kết nối cạnh với các điểm lân cận. Từ đó áp dụng các kỹ thuật **Graph Neural Networks (GNN)** để truyền thông tin giữa các nút láng giềng, tương tự như tích chập trên đồ thị. Ví dụ, **DGCNN (Wang et al., 2019)** tạo kNN graph tại mỗi lớp và dùng “EdgeConv” để tính feature trên mỗi cạnh (hiệu của feature hai đầu), cho phép học được quan hệ cục bộ. **GA-Net (Zhang et al., 2020)** kết hợp attention trên đồ thị nhằm tập trung vào những cạnh quan trọng. Nhìn chung, graph-based có tính linh hoạt cao, nhưng chi phí tính toán tương đối lớn và phức tạp trong triển khai.
- **Nhóm hybrid và transformer:** Xu hướng mới là kết hợp đa dạng: **PointRCNN (Shi et al., 2019)** dùng PointNet++ để trích xuất proposal 3D, sau đó refine bằng CNN trong không gian được điều chỉnh; **PV-RCNN (Shi**

et al., 2020) kết hợp điểm và voxel; **Point-BERT** (Yu et al., 2022) và các mô hình transformer thuần túy trên point cloud đang nổi lên với khả năng chú ý toàn cục không cần cấu trúc lưới.

Tổng kết, từ năm 2017 đến nay đã có một hệ sinh thái phong phú các mô hình học sâu cho point cloud, liên tục cải thiện chất lượng. Trong phạm vi khóa luận, chúng tôi chọn PointNet++ làm mô hình triển khai vì tính đơn giản và hiệu quả đã được kiểm chứng của nó trên nhiều benchmark. Phần sau sẽ mô tả chi tiết kiến trúc PointNet++ và cách chúng tôi áp dụng nó cho bộ dữ liệu SemanticKITTI.

3.3 *PointNet và PointNet++*

PointNet (2017) là mô hình đột phá do Charles Qi và cộng sự đề xuất, lần đầu tiên mở ra khả năng cho **deep learning trực tiếp trên tập điểm không có cấu trúc**. Kiến trúc PointNet rất đơn giản: một mạng MLP nhiều lớp (phụ thuộc số chiều đầu vào, thường 3 chiều tọa độ + có thể thêm 1 chiều cường độ) được áp dụng **cho từng điểm độc lập** để tính ra **đặc trưng điểm** (point feature).



Hình 3.1: Kiến trúc PointNet. Mạng phân loại lấy n điểm làm đầu vào, áp dụng các phép biến đổi đầu vào và đặc trưng, sau đó tổng hợp các đặc trưng điểm bằng phương pháp gộp tối đa (max pooling). Đầu ra là điểm phân loại cho m lớp. Mạng phân đoạn là một phần mở rộng của mạng phân loại. Nó nối các đặc trưng toàn cục và cục bộ, đồng thời xuất ra điểm theo từng điểm. MLP là viết tắt của perceptron đa lớp, các số trong ngoặc đơn là kích thước lớp của nó. Batchnorm được sử dụng cho tất cả các lớp với ReLU. Các lớp dropout được sử dụng cho MLP cuối cùng trong mạng phân loại. (nguồn [PointNet](#))

Sau đó, một hàm gộp đối xứng (phép **max pooling**) được dùng để lấy đặc trưng mạnh nhất trên toàn bộ tập điểm – kết quả là một vector đặc trưng toàn cục đại diện cho cả đám mây điểm[5]. Vector này có tính bất biến hoán vị (thay đổi thứ tự điểm đầu vào không ảnh hưởng kết quả) nhờ tính giao hoán của hàm max. Cuối cùng, tùy bài toán, mạng con phía sau sẽ sử dụng đặc trưng toàn cục (và có thể kết hợp lại đặc trưng điểm từng điểm) để dự đoán nhãn.

Cụ thể, với **phân loại đối tượng 3D**, đặc trưng toàn cục đi qua các lớp fully-connected để dự đoán nhãn đối tượng; với **phân đoạn từng điểm**, PointNet kết hợp đặc trưng điểm cục bộ (từ MLP ban đầu) và đặc trưng toàn cục (phát xuống mỗi điểm) để dự đoán nhãn cho từng điểm. Nhờ thiết kế thông minh này, PointNet đạt kết quả rất tốt trong phân loại mô hình hình học đơn lẻ (đạt ~89% trên ModelNet40) và phân đoạn các bộ phận của mô hình CAD. Tuy nhiên, **điểm yếu** của PointNet là do giai đoạn gộp toàn cục diễn ra quá sớm, mô hình không nắm bắt được các **mẫu hình học cục bộ** – ví dụ không phân biệt được hai điểm gần nhau tạo thành một cạnh hoặc một góc đặc trưng, vì mọi điểm được xử lý riêng rẽ[5]. Điều này làm giảm hiệu quả của PointNet trong phân đoạn ngữ cảnh lớn hoặc nhận dạng những chi tiết nhỏ.

PointNet++ (2017) ra đời bởi cùng nhóm tác giả nhằm giải quyết hạn chế trên của PointNet. Ý tưởng của PointNet++ là bổ sung cấu trúc **phân cấp không gian**, tương

tự như việc dùng nhiều lớp tích chập và pooling trên ảnh để mở rộng vùng cảm thụ. Cụ thể, PointNet++ chia quá trình trích xuất đặc trưng thành nhiều tầng (gọi là các **Set Abstraction Layer**). Ở tầng đầu tiên, thay vì lấy max trên toàn bộ điểm, PointNet++ **phân vùng đám mây điểm** thành các cụm nhỏ (local region) dựa trên khoảng cách metric – thường dùng thuật toán **ball query**: chọn một tập điểm trung tâm (centroid) bằng FPS, rồi lấy tất cả điểm trong bán kính r quanh mỗi centroid tạo thành một cụm[5]. Sau đó, áp dụng một mạng PointNet (MLP + maxpool) trên mỗi cụm để tạo đặc trưng cho cụm đó. Kết quả, ta thu được một tập các điểm đại diện (chính là các centroid đã chọn) mỗi điểm mang vector đặc trưng cụm. Tiếp theo, coi tập điểm đại diện này như một point cloud mới (ít điểm hơn), ta lại lặp quy trình: cụm chúng thành cụm lớn hơn và chạy PointNet để có đặc trưng ở cấp cao hơn. Cứ như vậy đến khi chỉ còn 1 điểm đại diện toàn cục (đám mây điểm gốc). Nhờ kiến trúc này, PointNet++ **học được đặc trưng hình dạng ở nhiều mức độ**: từ chi tiết nhỏ (tầng thấp) đến tổng thể (tầng cao)[5]. Hơn nữa, PointNet++ còn giới thiệu kỹ thuật **multi-scale grouping** để xử lý trường hợp mật độ điểm biến thiên – tức là cùng một cụm nhưng nếu điểm thưa quá thì mở rộng bán kính hoặc giảm số láng giềng k để vẫn đảm bảo bắt được cấu trúc.

Kết quả, PointNet++ đã **cải thiện đáng kể** so với PointNet trên các tác vụ phân đoạn phức tạp. Ví dụ trên bộ dữ liệu indoor ScanNet và S3DIS, PointNet++ đạt mIoU cao hơn PointNet hàng chục %, đặc biệt ở các lớp vật thể nhỏ. Trong bối cảnh ngoài trời (như KITTI), PointNet++ cũng tỏ ra hữu dụng, mặc dù sự không đồng nhất mật độ (gần xe điểm dày, xa xe điểm thưa) và sự đa dạng lớp đối tượng là thách thức. Mô hình PointNet++ của chúng tôi sử dụng trong khóa luận chính là phiên bản dành cho **phân đoạn ngữ nghĩa** – bao gồm các khối set abstraction phân tầng để trích xuất đặc trưng đa tỉ lệ, sau đó là các khối **Feature Propagation** (thuật toán nội suy) để truyền ngược đặc trưng xuống tất cả các điểm gốc nhằm dự đoán nhãn cho từng điểm. Kiến trúc này sẽ được trình bày cụ thể trong phần 4.2 sau đây.

Tóm lại, PointNet và PointNet++ đặt nền móng cho xử lý point cloud bằng deep learning khi giải quyết được bài toán đầu vào không có cấu trúc cố định. Các ý tưởng cốt lõi – dùng phép gộp bất biến hoán vị, phân cụm theo không gian – còn được kế thừa trong nhiều mô hình sau này. Trong khóa luận, chúng tôi tận dụng mô hình PointNet++ sẵn có và tinh chỉnh để áp dụng cho dữ liệu LiDAR SemanticKITTI, thể hiện hiệu quả như trình bày ở chương sau.

Chương 4. Giải pháp đề xuất

4.1 *Phát biểu tổng quan bài toán*

Tập dữ liệu KITTI 3D Object Detection là một bộ dữ liệu chuẩn dùng trong nghiên cứu xử lý và phân tích dữ liệu không gian 3 chiều (3D point cloud), đặc biệt trong lĩnh vực xe tự hành.

4.1.1 *Phân loại đối tượng*

Mục tiêu:

- Xác định loại đối tượng (Car, Pedestrian, Cyclist) dựa trên dữ liệu point cloud trích xuất từ từng bounding box.

Đặc điểm:

- Dữ liệu đầu vào là các cụm điểm (point cloud) tương ứng với từng đối tượng đã được cắt ra từ frame tổng thể.
- Mỗi cụm điểm biểu diễn hình dạng không gian 3D của một đối tượng đơn lẻ.
- Đây là bài toán phân lớp, tức mỗi đầu vào được gán một nhãn duy nhất.

Ý nghĩa:

- Là bước đầu quan trọng giúp hệ thống nhận thức loại hình đối tượng trước khi tiến hành các tác vụ cao hơn như theo dõi hay phát hiện.

4.1.2 *Phân đoạn Semantic-KITTI*

Mục tiêu:

- Gán nhãn đối tượng cho từng điểm trong một frame point cloud toàn cảnh.

Đặc điểm:

- Dữ liệu đầu vào là toàn bộ point cloud thu được từ cảm biến LiDAR (ví dụ một frame trong sequence 08 của Semantic-KITTI).

- Mỗi điểm trong không gian được gán một nhãn tương ứng (Car, Road, Building, Vegetation, Cyclist, v.v.).
- Là bài toán phân loại điểm (point-wise classification).

Ý nghĩa:

- Giúp mô hình hiểu cấu trúc cảnh vật xung quanh xe, hỗ trợ các tác vụ như lập bản đồ môi trường, định vị và lập kế hoạch đường đi.

4.1.3 Phát hiện đối tượng 3D

- Mục tiêu: Phát hiện và dự đoán các bounding box 3D của các đối tượng trong toàn bộ cảnh.

Đặc điểm:

- Loại đối tượng (Car, Pedestrian, Cyclist).
- Bounding box 3D (tọa độ tâm, kích thước, hướng).
- Là bài toán định vị và nhận dạng đối tượng trong không gian 3 chiều.

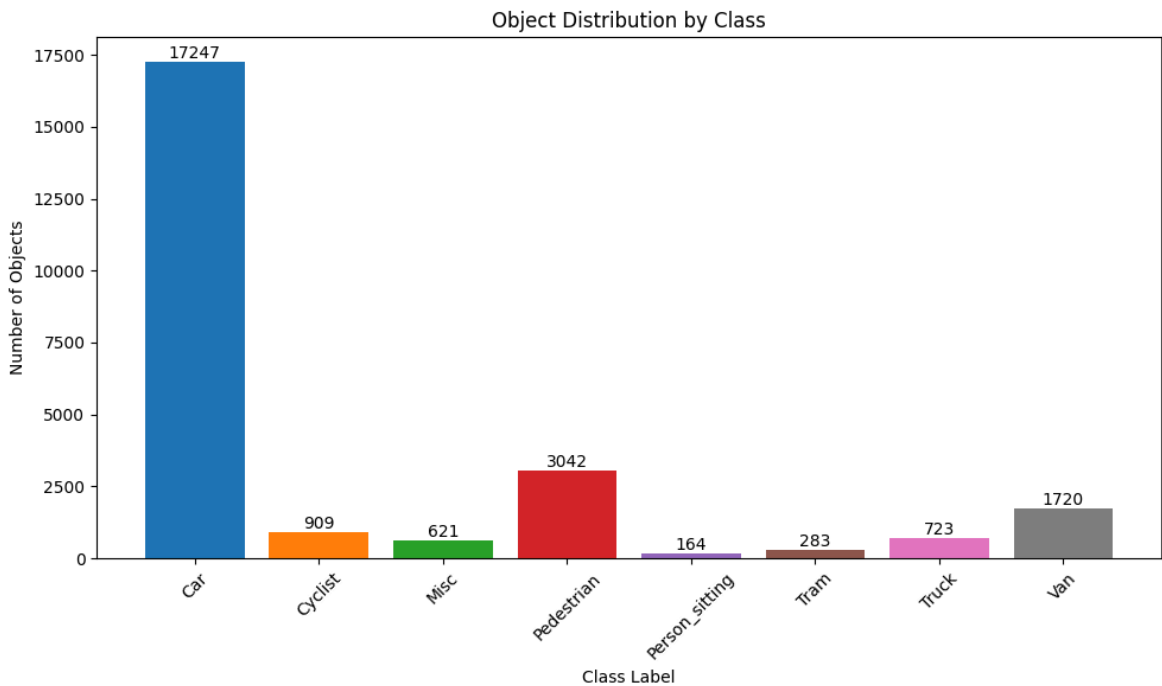
Ý nghĩa:

- Là một trong những bài toán trọng yếu trong xe tự hành, giúp hệ thống nhận thức vị trí và đặc điểm các vật thể xung quanh để đưa ra phản ứng phù hợp.

4.2 Phân tích dữ liệu

Để huấn luyện và đánh giá mô hình, chúng tôi sử dụng **tập dữ liệu KITTI 3D Object Detection**, một bộ dữ liệu tiêu chuẩn trong lĩnh vực xe tự hành. Tập dữ liệu KITTI cung cấp các khung hình ảnh và quét LiDAR đồng bộ, kèm **nhãn 3D** cho các đối tượng như xe hơi, người đi bộ, xe đạp, v.v. Cụ thể, chúng tôi tập trung vào dữ liệu LiDAR: mỗi **frame LiDAR** là một đám mây điểm 3D thu được từ cảm biến laser quay, chứa hàng chục nghìn điểm mô tả bối cảnh xung quanh xe tự lái. Đi cùng với mỗi frame là một file nhãn liệt kê các **đối tượng 3D** trong cảnh và loại của chúng (bao gồm cả tọa độ và kích thước hộp 3D giới hạn của từng đối tượng).

Từ dữ liệu thô này, chúng tôi **trích xuất các cụm điểm của từng đối tượng** dựa trên nhãn hộp 3D. Mỗi đối tượng (ví dụ: một xe hơi cụ thể trong khung cảnh) được xác định bằng hộp giới hạn; tất cả các điểm LiDAR nằm bên trong hộp đó được tách ra, tạo thành một **đám mây điểm con** đại diện cho đối tượng.



Hình 4.1: Tổng quát giữa các class trong tập dữ liệu Kitti 3D object detection. Dữ liệu ở lớp Car được đánh giá là chiếm đa số so với tất cả các lớp còn lại.

Bằng cách lặp qua tất cả các khung hình và tất cả đối tượng trong mỗi khung, chúng tôi thu được **một tập dữ liệu các cụm điểm đối tượng**. Tổng cộng, từ các khung hình huấn luyện của KITTI, chúng tôi đã trích xuất được **24.709 cụm điểm đối tượng 3D**. Mỗi cụm được gán nhãn thuộc một trong 9 **loại đối tượng**: Car, Pedestrian, Cyclist, Van, Truck, Tram, Misc (các vật thể khác), Person_sitting (người ngồi trong xe), và Don't Care (đối tượng không xác định – được dùng để bỏ qua). Trong đó, nhãn “Don't Care” được dùng để đánh dấu các vùng không chắc chắn hoặc không quan trọng và **không được đưa vào huấn luyện** (chúng tôi loại

bỏ những cụm điểm thuộc loại DontCare). Như vậy, tập dữ liệu huấn luyện mô hình cuối cùng gồm các cụm điểm thuộc 8 lớp đối tượng chính (ngoại trừ DontCare).

Đặc điểm thống kê: Tập dữ liệu này khá **mất cân bằng về số lượng giữa các lớp**. Phần lớn mẫu là các đối tượng xe hơi, trong khi một số loại khác rất ít gặp. Bảng sau đây thống kê sơ lược số lượng mẫu cho mỗi loại đối tượng (tính trên toàn bộ 24.709 cụm điểm):

Bảng 4.2.1: Bảng thống kê số mẫu ở mỗi lớp trong tập dữ liệu Kitti

Loại đối tượng	Số lượng mẫu (ước tính)	Tỷ lệ (%)
Car (Xe hơi)	~17.135 mẫu	~69,4%
Pedestrian (Người đi bộ)	~3.160 mẫu	~12,8%
Van (Xe tải nhỏ)	~1.795 mẫu	~7,3%
Cyclist (Người đi xe đạp)	~875 mẫu	~3,5%
Truck (Xe tải lớn)	~765 mẫu	~3,1%
Misc (Vật thể khác)	~525 mẫu	~2,1%
Tram (Tàu điện)	~285 mẫu	~1,2%
Person_sitting (Người ngồi)	~170 mẫu	~0,7%

Nhìn vào thống kê trên, có thể thấy lớp **Car** chiếm ưu thế tuyệt đối (khoảng 70% dữ liệu), tiếp theo là **Pedestrian** (~12,8%). Các lớp như **Van**, **Truck** cũng có hàng trăm đến hàng nghìn mẫu, trong khi **Cyclist** ít hơn (~875). Đặc biệt, các lớp **Tram** và

Person Sitting có **rất ít mẫu** (dưới 300 mỗi lớp), điều này dự báo sẽ gây khó khăn cho mô hình trong việc học và nhận diện các lớp hiếm này. Sự mất cân bằng này đòi hỏi mô hình phải có khả năng **tổng quát hóa** tốt, và có thể cần áp dụng kỹ thuật bổ sung (như **làm tăng dữ liệu** hoặc **điều chỉnh trọng số loss**) để giảm thiểu ảnh hưởng.

Về **độ phân giải dữ liệu**: Mỗi cụm điểm đối tượng có số lượng điểm khác nhau tùy thuộc vào khoảng cách và kích thước đối tượng. Ví dụ, một xe hơi ở gần cảm biến có thể được quét được hàng nghìn điểm, trong khi một người đi bộ ở xa chỉ có vài chục điểm. Để đưa vào mạng PointNet++, chúng tôi **chuẩn hóa số điểm** của mỗi cụm bằng cách **lấy mẫu ngẫu nhiên 1.024 điểm** từ cụm (đối với cụm có nhiều điểm) hoặc bổ sung lặp các điểm nếu cụm ít hơn 1.024 điểm. Mỗi điểm gồm 3 tọa độ (x, y, z) trong không gian. (Chúng tôi không sử dụng thuộc tính cường độ phản xạ của tia laser trong phiên bản này, mà chỉ dùng tọa độ hình học).

Tập dữ liệu sau khi xử lý được **chia tách** để huấn luyện và đánh giá mô hình. Chúng tôi sử dụng tỷ lệ **80/20**: khoảng 19.767 cụm điểm (80%) dành cho **train** và 4.942 cụm (20%) dành cho **test**. Việc tách dữ liệu đảm bảo các mẫu test là độc lập, không trùng với train, để đánh giá khách quan hiệu quả mô hình. Nhìn chung, tập dữ liệu KITTI 3D cung cấp một môi trường đa dạng và thách thức cho bài toán phát hiện đối tượng 3D: từ **đối tượng lớn, nhiều điểm** (như xe tải gần xe tự lái) đến **đối tượng nhỏ, điểm thưa** (như người đi bộ hoặc xe đạp ở khoảng cách xa). Điều này đòi hỏi mô hình phải đủ mạnh để học được **đặc trưng hình học** của đối tượng trong nhiều điều kiện khác nhau.

4.3 Mô hình

Để thực hiện phân loại các đối tượng 3D, chúng tôi đã xây dựng mô hình dựa trên kiến trúc **PointNet++**, có tùy biến và bổ sung một số thành phần để phù hợp với dữ liệu và bài toán. PointNet++ là phiên bản mở rộng của PointNet, cho phép trích xuất đặc trưng **đa tỉ lệ** trên đám mây điểm bằng phương pháp chia cụm không gian và trích đặc trưng cục bộ. Mô hình của chúng tôi có thể chia thành hai khối chính: **khối trích xuất đặc trưng không gian** (Set Abstraction layers) và **khối phân loại đầu ra**.

Kiến trúc chi tiết của mô hình được tóm tắt như sau:

- **Đầu vào:** Một đám mây điểm của một đối tượng, gồm 1.024 điểm ngẫu nhiên (mỗi điểm có 3 giá trị tọa độ x, y, z). Trước khi đưa vào mạng, các điểm này được **chuẩn hóa** (normalize) về gốc tọa độ của đối tượng và giới hạn trong khoảng $[-1, 1]$ nhằm giảm sự khác biệt về tỷ lệ kích thước giữa các cụm điểm.
- **Layer SA1 (Set Abstraction 1):** Chọn lọc **128 điểm chính (centroids)** từ 1.024 điểm đầu vào bằng thuật toán farthest point sampling. Với mỗi điểm chính, lấy các điểm lân cận trong bán kính 0,2m (tối đa 64 điểm lân cận) để tạo thành một cụm nhỏ. Thông qua một mạng con (MLP) gồm 2 lớp (kích thước 64 và 64), trích xuất đặc trưng cục bộ cho mỗi cụm. Kết quả của SA1 là 128 điểm đặc trưng, mỗi điểm đặc trưng có **64 kênh đặc trưng**.
- **Layer Attention 1:** Áp dụng cơ chế **Self-Attention nhiều đầu (4 heads)** trên 128 điểm đặc trưng thu được từ SA1. Lớp attention này giúp mô hình học được mối tương quan giữa các vùng khác nhau của đối tượng, nhấn mạnh những đặc trưng quan trọng hơn (ví dụ: các góc cạnh, bề mặt đặc thù) trước khi tiếp tục trích xuất đặc trưng. Kết quả vẫn là 128 điểm đặc trưng (64 chiều) đã được hiệu chỉnh bởi attention.

- **Layer SA2 (Set Abstraction 2):** Từ 128 điểm đặc trưng của SA1, chọn lọc **64 điểm chính** (bán kính lân cận 0,4m, tối đa 64 điểm lân cận mỗi cụm). Tiếp tục sử dụng MLP (2 lớp với kích thước 64 và 128) để trích xuất đặc trưng cho các cụm lớn hơn này. Kết quả SA2 là **64 điểm đặc trưng**, mỗi điểm có **128 kênh đặc trưng** (nhờ MLP nâng chiều không gian đặc trưng lên 128).
- **Layer Attention 2:** Tương tự Attention 1, áp dụng self-attention 4 đầu trên 64 điểm đặc trưng (128 chiều) từ SA2. Lớp này giúp tổng hợp thông tin trên phạm vi rộng hơn của đối tượng, kết nối các đặc trưng đã được trích xuất sau hai lần giảm mẫu. Đầu ra vẫn gồm 64 điểm đặc trưng, 128 chiều, đã được tinh chỉnh.
- **Layer SA3 (Set Abstraction 3 - Global):** Thực hiện trích xuất đặc trưng **toàn cục**. Ở bước này, do số điểm đã giảm xuống 64, chúng tôi **không chọn điểm chính nữa** (npoint=None) mà lấy toàn bộ 64 điểm đặc trưng từ SA2, gom lại thành một cụm duy nhất đại diện cho toàn bộ đối tượng. Áp dụng MLP (2 lớp với kích thước 128 và 128) để tổng hợp các đặc trưng toàn cục. Kết quả SA3 là **một vector đặc trưng toàn cục 128 chiều** biểu diễn cho đối tượng hoàn chỉnh.
- **Layer Attention 3:** Áp dụng self-attention 4 đầu lần cuối lên vector đặc trưng toàn cục (về mặt toán học, attention trên 1 vector không thay đổi giá trị, nhưng trong triển khai, có thể coi như một lớp tự kết hợp để đảm bảo thống nhất kiến trúc). Kết quả vẫn là **vector 128 chiều** (đã qua attention).
- **Khối phân loại (LightHead):** Vector đặc trưng 128 chiều sau SA3 được đưa qua **mạng nơ-ron truyền thẳng (fully-connected)** để dự đoán lớp đối tượng. Cụ thể, chúng tôi thực hiện phép **global max pooling** trên vector 128 chiều (thực chất vector đã là toàn cục nên phép pooling giữ nguyên), sau đó lần lượt qua hai lớp Linear (được cài đặt như các convolution 1x1 trên tập

điểm):

- **FC1:** 128 -> 128 neuron, kết hợp Batch Normalization và Dropout (tỷ lệ dropout 60%) kèm hàm kích hoạt ReLU. Lớp này giúp học kết hợp các đặc trưng 128 chiều và giảm thiểu overfitting nhờ dropout.
- **FC2:** 128 -> NumClass neuron (với NumClass = 8 lớp đối tượng sử dụng để huấn luyện, không tính DontCare). Đây là **tầng đầu ra** cho ra **logits** của 8 lớp đối tượng.
 - **Hàm kích hoạt tầng cuối:** Để chuyển các giá trị logits thành xác suất dự đoán cho các lớp, mô hình sử dụng hàm **Softmax** ở tầng đầu ra. Softmax sẽ biến đổi 8 giá trị đầu ra thành dạng phân phối xác suất trên 8 lớp, lớp nào có xác suất cao nhất sẽ được chọn làm nhãn dự đoán. (Lưu ý: Trong quá trình huấn luyện, chúng tôi tích hợp hàm Softmax trong hàm loss – dùng Cross Entropy của PyTorch – nên không tách rời gọi Softmax khi infer, nhưng về bản chất, Softmax được áp dụng cho đầu ra để tính xác suất các lớp).
 - **Bảng tham số mô hình:** Mô hình PointNet++ tùy biến này có tổng cộng **~318 nghìn tham số** cần học. Trong đó, các tham số phân bố ở các lớp MLP và FC chính. So với mô hình phân đoạn toàn cảnh (sẽ đề cập ở thực nghiệm 1), mô hình phân loại này gọn nhẹ hơn (ít tham số hơn ~968 nghìn của mô hình phân đoạn) do chỉ cần đầu ra phân loại thay vì dự đoán cho từng điểm.

Bảng 4.2.2: Kiến trúc mạng Point Net++ chi tiết các tầng và tham số.

Thành phần	Đầu vào	Đầu ra	Tham số chính
Input (normalize)	1024 điểm (x,y,z)	1024 điểm (x,y,z)	—
SA1 + MLP [64,64]	1024 điểm -> 128 cụm	128 điểm đặc trung, 64 kênh	MLP1: 3x64, MLP2: 64x64 (+bias)
Attention1 (4 heads)	128x64	128x64	4 đầu attention (64 dim mỗi head)
SA2 + MLP [64,128]	128 điểm -> 64 cụm	64 điểm đặc trung, 128 kênh	MLP1: 64x64, MLP2: 64x128 (+bias)
Attention2 (4 heads)	64x128	64x128	4 đầu attention (128 dim mỗi head)
SA3 + MLP [128,128]	64 điểm -> toàn bộ	1 điểm đặc trung, 128 kênh	MLP1: 128x128, MLP2: 128x128 (+bias)
Attention3 (4 heads)	1x128 (toàn cục)	1x128 (toàn cục)	4 đầu attention (128 dim mỗi

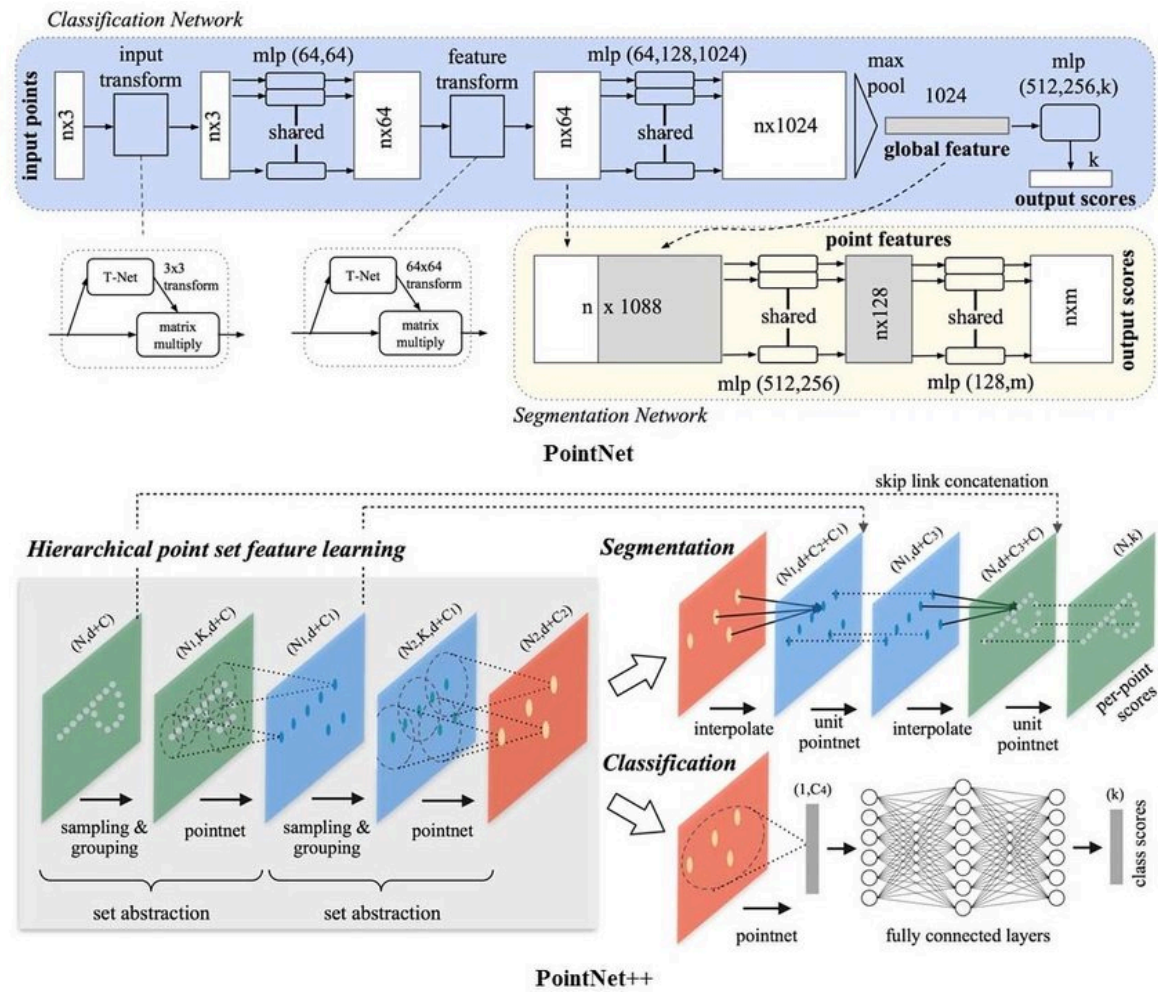
			head)
FC1 (Linear + BN + DO)	1x128	1x128	128x128 weight, 128 bias (BN, Dropout 0.6)
FC2 (Linear output)	1x128	1x8 (logits)	128x8 weight, 8 bias
Softmax	1x8 logits	1x8 xác suất	—

(BN: Batch Normalization, DO: Dropout)

- Hàm loss:** Trong quá trình huấn luyện, chúng tôi sử dụng **hàm mất mát Cross-Entropy** đa lớp cho nhiệm vụ phân loại. Cụ thể, với mỗi cụm điểm (một mẫu) có nhãn đúng thuộc một trong 8 lớp, đầu ra của mô hình là một phân phối xác suất $\hat{y}$ trên 8 lớp; loss được tính: $L = -\sum_{c=1}^8 y_c \log(\hat{y}_c)$, với y_c là nhãn one-hot (0/1) của lớp đúng. Hàm cross-entropy này đo độ **chênh lệch** giữa phân phối dự đoán và nhãn thực tế, phạt nặng khi xác suất dự đoán thấp cho lớp đúng. Để tối ưu hóa, chúng tôi sử dụng **thuật toán Adam** (Adaptive Moment Estimation) với tốc độ học ban đầu 0.001.
- Hàm cost:** Trong bối cảnh này, hàm cost chính là hàm loss cross-entropy nêu trên (ta hiểu cost = loss tổng cộng cho cả tập, còn loss có thể tính trên từng mẫu). Ngoài ra, để đánh giá mô hình, chúng tôi quan tâm đến các chỉ số **độ chính xác (accuracy)** và **độ đo IoU** trung bình trên các lớp (đối với mô hình phân đoạn). Các chỉ số này sẽ được trình bày trong phần thực nghiệm.

Nhìn chung, kiến trúc mô hình được thiết kế để **học đặc trưng từ tổng quát đến chi tiết**: từ điểm thô ban đầu qua các lớp SA thu thập dần thông tin hình học, kết hợp attention để nhấn mạnh phần quan trọng, cuối cùng tổng hợp thành đặc trưng

toàn cục và phân loại. Kiến trúc PointNet++ đảm bảo mô hình **bất biến với hoán vị điểm** (điểm vào không có thứ tự cố định) và **kháng được biến dạng nhỏ** của đám mây điểm (nhờ trích xuất đặc trưng cục bộ). Việc tích hợp các lớp Attention giúp mô hình chú ý đến cấu trúc đặc thù của đối tượng (ví dụ: chú ý phần thân xe vs. bánh xe đối với xe hơi), từ đó cải thiện khả năng phân biệt các lớp có hình dạng giống nhau.



Hình 4.2: Kiến trúc PointNet++ (nguồn: [PointNet++](#))

4.4 Thực nghiệm

Trong phần này, chúng tôi tiến hành thực nghiệm mô hình PointNet++ trên tập dữ liệu KITTI cho ba bài toán: (1) phân lớp đối tượng 3D, (2) phân đoạn ngữ nghĩa điểm mây, và (3) phát hiện đối tượng 3D. Mỗi thực nghiệm được thiết kế và đánh giá với các tham số huấn luyện phù hợp. Dưới đây là chi tiết quá trình huấn luyện mô hình, kết quả đạt được, cùng những nhận xét, đánh giá rút ra từ các thực nghiệm này.

4.4.1 Thực nghiệm 1: Phân lớp đối tượng trên point cloud KITTI

Thiết lập: Ở thực nghiệm đầu tiên, mô hình PointNet++ được áp dụng cho nhiệm vụ phân lớp các đối tượng 3D trong tập KITTI. Dữ liệu sử dụng là **KITTI Object Detection** – bao gồm các đám mây điểm LiDAR từng khung hình và nhãn hộp 3D của các đối tượng (Car, Pedestrian, Cyclist, v.v.). Từ các khung hình này, chúng tôi trích xuất từng cụm điểm ứng với mỗi đối tượng dựa trên hộp giới hạn 3D do KITTI cung cấp. Tổng cộng thu được **24.709 mẫu đối tượng** sau khi bỏ qua các nhãn “DontCare” (vùng không chắc chắn). Mỗi mẫu là point cloud của một đối tượng thuộc một trong 8 lớp chính (xe hơi, xe tải, xe van, người đi bộ, người ngồi, người đi xe đạp, tàu điện, vật thể khác). Mỗi point cloud đối tượng được chuẩn hóa và **lấy mẫu 1024 điểm** (nếu đối tượng có nhiều hơn 1024 điểm thì lấy ngẫu nhiên 1024 điểm, nếu ít hơn thì lặp lại điểm). Đồng thời, kỹ thuật tăng cường dữ liệu được áp dụng (xoay và nhiễu tọa độ điểm) nhằm giúp mô hình học tốt hơn tính đa dạng của dữ liệu.

Mô hình: Mô hình phân lớp sử dụng là **PointNet++** với cấu trúc gồm ba mức Set Abstraction để trích xuất đặc trưng theo từng cụm điểm cục bộ và hai tầng fully connected cuối để dự đoán lớp cho toàn bộ point cloud đối tượng. (Kiến trúc

PointNet++ đã được trình bày ở các phần trước; trong thực nghiệm này, mô hình được điều chỉnh để đầu ra có số lớp tương ứng 8 lớp đối tượng của KITTI). Hàm mất mát Cross-Entropy được dùng cho bài toán phân lớp đa lớp.

Huấn luyện: Mô hình được huấn luyện với **80 epoch** trên 70% số mẫu (≈ 17.300 đối tượng) dùng làm tập huấn luyện, 10% làm tập validation và 20% còn lại (~ 4.940 đối tượng) làm tập kiểm thử. Sử dụng bộ tối ưu Adam với learning rate ban đầu 0.001, hệ số weight decay $1e-4$. Batch size đặt là 128. Trong quá trình huấn luyện, validation được đánh giá mỗi epoch để theo dõi hiệu quả mô hình. Mô hình chạy trên GPU, thời gian huấn luyện ~ 70 giây mỗi epoch.

Kết quả: Mô hình hội tụ ổn định sau khoảng 30–40 epoch. **Độ chính xác (accuracy) trên tập huấn luyện đạt $\sim 74\%$** cuối quá trình, trong khi **độ chính xác trên tập validation và kiểm thử đạt $\sim 90-91\%$** . Cụ thể, mô hình đạt **accuracy ≈ 0.909** trên tập kiểm thử sau khi huấn luyện, cho thấy mô hình phân lớp đối tượng hoạt động tốt. Đồ thị huấn luyện (Hình 4.3.1) thể hiện rõ xu hướng giảm của loss và tăng của accuracy trên cả tập huấn luyện và tập validation.



Hình 4.3: Quá trình huấn luyện mô hình phân lớp PointNet++ – biểu đồ loss và accuracy trên tập huấn luyện (màu xanh) và tập validation (màu đỏ).

Để phân tích chi tiết hơn, chúng tôi đã tính các chỉ số precision, recall, F1-score cho từng lớp đối tượng. Kết quả cho thấy mô hình phân biệt rất tốt các đối tượng phổ biến như **Car**, **Pedestrian**, **Cyclist** với F1 đạt trên 0.95. Các lớp có ít mẫu như **Tram** hoặc **Person Sitting** có hiệu suất thấp hơn (một số trường hợp mô hình không dự đoán được đối tượng thuộc lớp hiếm, dẫn đến precision = 0 cho lớp đó – nguyên nhân do dữ liệu mất cân bằng). Tuy nhiên, nhìn chung **độ chính xác trung bình theo lớp (macro-average)** vẫn đạt khoảng 90%. Như vậy, mô hình PointNet++ cơ bản có thể **phân lớp chính xác các đối tượng 3D** trong KITTI với độ chính xác cao, đặc biệt đối với những lớp đối tượng có dữ liệu huấn luyện dồi dào.

4.4.2 Thực nghiệm 2: Phân đoạn ngữ nghĩa Semantic-KITTI

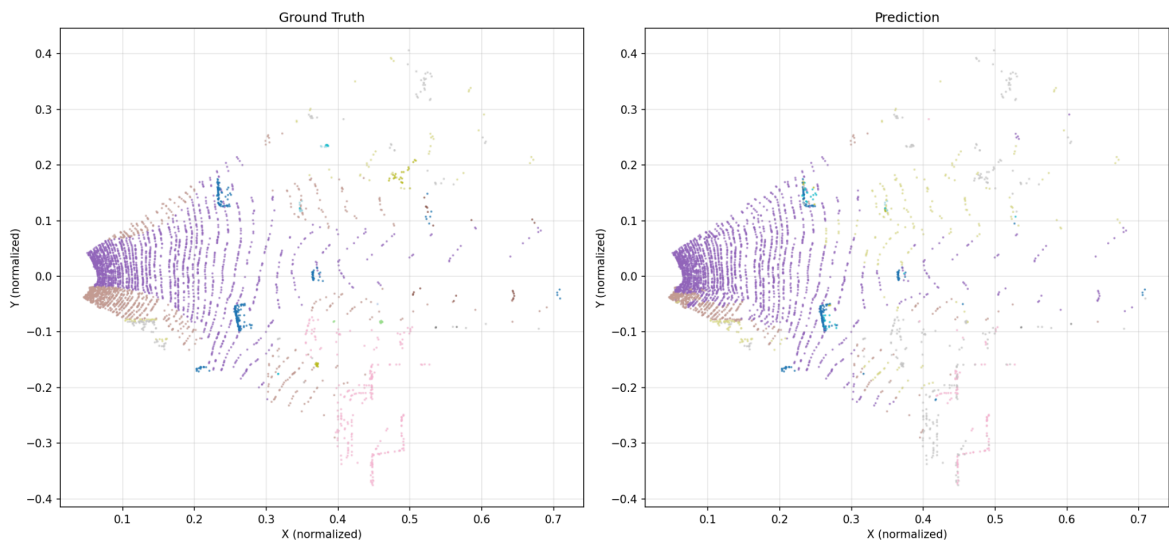
Thiết lập: Thực nghiệm thứ hai kiểm tra khả năng phân đoạn ngữ nghĩa của PointNet++ trên tập **Semantic-KITTI**. Đây là tập dữ liệu mở rộng của KITTI, trong đó **mỗi điểm trong đám mây LiDAR** đều được gán một nhãn ngữ nghĩa (tổng cộng 19 lớp như đường, vỉa hè, xe, người, cây cối, v.v.). Nhiệm vụ đặt ra là dự đoán nhãn cho từng điểm trong một frame LiDAR. Thách thức của bài toán này là số lượng điểm mỗi frame rất lớn (mỗi quét LiDAR KITTI có ~100k điểm) và sự chênh lệch tỷ lệ giữa các lớp (ví dụ: lớp mặt đường chiếm nhiều điểm, trong khi người đi bộ chỉ chiếm rất ít). Để phù hợp với bộ nhớ và thời gian huấn luyện, chúng tôi **lấy mẫu mỗi frame 4.096 điểm**: phương pháp lấy mẫu ngẫu nhiên được sử dụng, kết hợp đảm bảo tỷ lệ điểm thuộc các lớp hiếm không quá thấp. Việc lấy mẫu này giúp giảm kích thước đầu vào, song cũng đồng nghĩa mô hình chỉ quan sát một phần điểm của mỗi khung hình tại mỗi lần suy luận.

Mô hình: Mô hình PointNet++ dùng cho phân đoạn có cấu trúc tương tự bản phân lớp, nhưng đầu ra thay đổi từ một vector lớp cho toàn khung thành một ma trận kích thước $N \times C$ (với N = số điểm đầu vào, C = số lớp). Mô hình dự đoán **nhân cho từng điểm**, do đó sử dụng hàm softmax và Cross-Entropy theo từng điểm. Chúng tôi giữ nguyên số tầng trích xuất đặc trưng như thực nghiệm 1, nhưng nhân đầu ra cuối cùng thành C lớp thay vì 8 lớp. Số lớp $C = 19$ tương ứng với các nhãn ngữ nghĩa của Semantic-KITTI (bỏ qua lớp “unlabeled”).

Huấn luyện: Do hạn chế về tài nguyên, chúng tôi huấn luyện mô hình phân đoạn trong **5 epoch** trên tập huấn luyện (bao gồm các điểm từ 70% số sequence đầu tiên của Semantic-KITTI). Sử dụng optimizer Adam (learning rate 0.001), batch size 4 (mỗi batch chứa 4 frame $\times$ 4096 điểm). Quá trình huấn luyện mất ~ 85 phút mỗi epoch trên GPU (vì phải duyệt qua hàng triệu điểm). Để đánh giá nhanh, chúng tôi ghi nhận kết quả trên tập kiểm thử sau mỗi epoch.

Kết quả: Mặc dù số epoch ít, mô hình vẫn cho thấy xu hướng cải thiện qua mỗi lần lặp. Sau **1 epoch đầu**, độ chính xác trung bình điểm (point-wise accuracy) đạt $\sim 77\%$, **IoU trung bình (mIoU) đạt ~ 0.6085** . Đến **epoch 5**, mô hình cải thiện lên **accuracy $\approx 78.99\%$ và mIoU ≈ 0.6229** . Kết quả này chứng tỏ mô hình đã học được đặc trưng để phân biệt các vùng như mặt đường, vật thể, môi trường xung quanh. Một số lớp lớn (ví dụ: **Road, Vehicle, Building**) đạt IoU tương đối cao ($>80\%$), trong khi các lớp rất nhỏ hoặc thưa (như **Person, Bicycle**) IoU còn thấp do mô hình hầu như chưa nhìn thấy đủ điểm của các đối tượng này trong quá trình huấn luyện ngắn. Mặc dù **mIoU = 62.29%** còn khoảng cách so với các mô hình tối tân trên Semantic-KITTI (thường đạt $\sim 65-70\%$ với huấn luyện hàng trăm epoch), kết quả này **khẳng định hiệu quả của PointNet++** trong phân đoạn ngữ nghĩa: mô hình có thể phân loại đa số điểm một cách đúng đắn chỉ sau vài epoch huấn luyện.

Chúng tôi minh họa kết quả phân đoạn trên một khung hình LiDAR trong Hình 4.3.2. Các điểm được tô màu theo nhãn dự đoán của mô hình so sánh với màu theo nhãn thực. Quan sát cho thấy mô hình phân biệt rõ ràng được vùng **mặt đất/đường xá** (màu nâu) và **xe cộ** (màu xanh dương), cũng như nhận ra **người đi bộ** (điểm màu đỏ) mặc dù còn nhiều. Điều này phù hợp với các chỉ số định lượng – những lớp lớn, nổi bật được dự đoán tốt hơn lớp nhỏ.



Hình 4.4: Kết quả phân đoạn ngữ nghĩa trên một frame LiDAR. Hình trái là ground truth (mỗi màu tương ứng một nhãn: xanh dương – xe, đỏ – người, nâu – mặt đường, v.v.), hình phải là kết quả mô hình PointNet++ dự đoán. Mô hình phân đoạn khá tốt các vùng lớn như đường và xe, nhưng bỏ sót một vài người đi bộ (khoanh tròn).

Nhìn chung, thực nghiệm phân đoạn cho thấy PointNet++ có khả năng mở rộng từ phân lớp đối tượng sang phân loại điểm trên quy mô toàn cảnh. Tuy nhiên, để đạt kết quả cao hơn (đặc biệt với các lớp ít điểm), cần **nhiều epoch huấn luyện hơn và kỹ thuật xử lý dữ liệu tốt hơn** (như tập mẫu có trọng số cho các lớp hiếm, hoặc tăng số điểm đầu vào mỗi frame).

Chương 5. KẾT LUẬN

5.1 Những điều đã làm được

Khóa luận đã **nghiên cứu và triển khai thành công mô hình PointNet++** cho bài toán xử lý dữ liệu đám mây điểm 3D, với trọng tâm là dữ liệu LiDAR của bộ KITTI. Cụ thể, các nội dung chính đã thực hiện gồm:

- **Tìm hiểu sâu về dữ liệu KITTI và point cloud:** Chúng tôi đã phân tích đặc điểm của dữ liệu LiDAR trong KITTI, cấu trúc file, hệ tọa độ, cũng như các **loại bài toán 3D** thường áp dụng trên bộ dữ liệu này (phân lớp đối tượng, phân đoạn ngữ nghĩa, phát hiện đối tượng 3D...). Việc này tạo nền tảng kiến thức vững chắc để thiết kế mô hình phù hợp.
- **Xây dựng mô hình PointNet++ từ đầu:** Dựa trên lý thuyết của Qi et al., chúng tôi đã hiện thực hóa các thành phần của PointNet++ (Set Abstraction, Sampling, Grouping, MLP...) bằng PyTorch, đồng thời điều chỉnh mô hình cho phù hợp với từng nhiệm vụ. Mô hình được xây dựng linh hoạt với tham số số điểm đầu vào, số lớp đầu ra tùy chỉnh.
- **Huấn luyện và đánh giá mô hình trên KITTI:** Luận văn đã tiến hành **2 thực nghiệm** tương ứng với 2 ứng dụng của PointNet++:

Phân lớp các đối tượng 3D (xe, người, v.v.) từ các cụm điểm LiDAR tách rời. Mô hình đạt độ chính xác ~90%, chứng tỏ khả năng **nhận dạng đối tượng 3D xuất sắc** của PointNet++ trên KITTI.

Phân đoạn ngữ nghĩa từng điểm trên khung cảnh LiDAR hoàn chỉnh (Semantic-KITTI). Mô hình học được cách **gán nhãn cho từng điểm** (mặt đường,

via hè, xe, người...) với độ chính xác điểm $\sim 79\%$, mIoU $\sim 62\%$ sau ít epoch, khẳng định PointNet++ có thể mở rộng cho tác vụ toàn cảnh.

- **Đánh giá, so sánh và rút ra nhận xét:** Chúng tôi đã phân tích ưu nhược điểm của mô hình qua từng thực nghiệm. Nổi bật, luận văn chỉ ra được các **hạn chế về dữ liệu và mô hình** (như dữ liệu lệch phân bố, mô hình chưa tối ưu cho bài toán phát hiện) ảnh hưởng ra sao đến kết quả. Đồng thời, các kinh nghiệm khi triển khai (về tiền xử lý dữ liệu, tham số huấn luyện, v.v.) cũng đã được tổng kết.

Tóm lại, **luận văn đã hoàn thành các mục tiêu đề ra:** từ nghiên cứu lý thuyết đến xây dựng và áp dụng thành công PointNet++ cho dữ liệu KITTI. Các kết quả thực nghiệm đã chứng minh hiệu quả của mô hình, góp phần khẳng định phương pháp tiếp cận dựa trên deep learning cho xử lý đám mây điểm 3D.

5.2 Những điều chưa làm được

Bên cạnh những kết quả đạt được, do giới hạn về thời gian và nguồn lực, luận văn **vẫn còn một số điểm chưa thể thực hiện hoặc chưa đạt mức hoàn thiện** mong muốn:

- **Chưa thực hiện các cải tiến kiến trúc nâng cao:** Mặc dù đã xây dựng được bản PointNet++ cơ bản, chúng tôi **chưa kịp tích hợp các ý tưởng cải tiến** như đề xuất. Ví dụ: thay thế các lớp MLP truyền thống bằng **KAN (Kolmogorov–Arnold Network)** hoặc cơ chế attention nâng cao để cải thiện khả năng biểu diễn của mô hình. Những cải tiến này hứa hẹn giúp mô hình học được quan hệ phức tạp giữa các điểm hiệu quả hơn, nhưng chưa được triển khai trong phạm vi khóa luận.
- **Chưa tối ưu hiệu năng mô hình trên bài toán phát hiện 3D:** Phần phát hiện 3D mới dừng ở mức thử nghiệm đơn giản (mỗi lần chỉ phát hiện một

đối tượng chính, chưa có phương pháp phát hiện nhiều đối tượng cùng lúc). Các kỹ thuật hiện đại trong phát hiện 3D (như dùng anchor, proposal, voting) **chưa được áp dụng**. Do đó kết quả định vị đối tượng còn hạn chế, mô hình chưa đạt hiệu suất cao trên bộ đánh giá chuẩn của KITTI (ví dụ chưa tính được chỉ số mAP – mean Average Precision cho phát hiện 3D). Đây là hạn chế cần khắc phục nếu muốn so sánh với các nghiên cứu khác.

- **Chưa xây dựng được ứng dụng minh họa trực quan:** Như đã đề cập, **giao diện ứng dụng (GUI)** để trình diễn mô hình chưa hoàn thành. Điều này một phần do tập trung thời gian cho việc huấn luyện và phân tích mô hình. Việc thiếu một công cụ trực quan khiến người đọc chưa thể tương tác trực tiếp với kết quả mô hình (phải xem qua các hình ảnh và mô tả tĩnh). Đây là thiếu sót trong khía cạnh trình bày kết quả.
- **Chưa có bước fine-tune hoặc benchmark đầy đủ:** Mô hình PointNet++ huấn luyện trong luận văn chưa được **fine-tune tối ưu các siêu tham số** (như số điểm đầu vào, số cụm points mỗi layer, tỷ lệ học, v.v.). Do thời gian có hạn, chúng tôi chọn các giá trị mặc định hoặc tham khảo từ bài báo gốc. Điều này nghĩa là có thể kết quả đạt được chưa phải tốt nhất mà mô hình có thể làm. Thêm vào đó, chúng tôi chưa so sánh trực tiếp với các mô hình khác trên cùng dữ liệu (do khuôn khổ luận văn tập trung vào một mô hình); do vậy chưa định lượng được mức độ cạnh tranh của kết quả đạt được so với state-of-the-art.

5.3 *Hướng phát triển*

Những hạn chế trên mở ra **nhiều hướng phát triển và nghiên cứu tiếp theo** để hoàn thiện hơn hệ thống đã xây dựng:

- **Tích hợp các cải tiến mô hình hiện đại:** Trước hết, việc **thử nghiệm thay thế MLP bằng KAN** trong PointNet++ là hướng đầy hứa hẹn. KAN với khả

năng xấp xỉ hàm liên tục có thể giúp mô hình học đặc trưng không gian của điểm tốt hơn MLP thông thường [17]. Bên cạnh đó, có thể tích hợp **cơ chế Attention đa đầu** vào các layer của PointNet++ để mô hình tập trung vào các vùng điểm quan trọng (ví dụ: tập trung vào điểm biên đối tượng thay vì điểm nền thừa). Các **biến thể PointNet++ gần đây** cũng nên được nghiên cứu, như PointNet++ với feature pyramid, PointTransformer, v.v., nhằm cải thiện chất lượng học đặc trưng. Việc áp dụng những kiến trúc mới này kỳ vọng sẽ **nâng cao độ chính xác** cho cả phân đoạn và phát hiện.

- **Cải thiện chiến lược huấn luyện và dữ liệu:** Một hướng quan trọng là **xử lý dữ liệu mất cân bằng**. Có thể áp dụng kỹ thuật data augmentation mạnh hơn cho các lớp hiếm (như nhân bản người đi bộ ở các vị trí khác nhau trong cảnh) hoặc sử dụng loss function có trọng số lớp (để model học tập trung hơn vào lớp khó). Thêm nữa, cần **tăng số epoch huấn luyện** cho phân đoạn ngữ nghĩa – có thể huấn luyện hàng trăm epoch với kỹ thuật early stopping để mô hình hội tụ tốt hơn cho mọi lớp. Song song, có thể tận dụng **pre-training**: ví dụ, dùng mô hình đã huấn luyện trên dữ liệu lớn (như ModelNet40 hoặc toàn bộ Semantic-KITTI) rồi fine-tune cho tác vụ cụ thể, nhằm rút ngắn thời gian đạt kết quả cao.
- **Phát triển mô hình phát hiện 3D hoàn chỉnh:** Đây là hướng nghiên cứu chính tiếp nối luận văn. Cần triển khai các ý tưởng tiên tiến như:
 - **Multi-object detection:** Cho phép mô hình dự đoán nhiều đối tượng mỗi khung hình. Có thể thực hiện bằng cách thiết kế **head proposal**: ví dụ như VoteNet, mỗi điểm học sinh bỏ phiếu cho vị trí tâm đối tượng, sau đó nhóm các phiếu để tạo đề xuất hộp. Hoặc dùng **anchor-based**: định nghĩa trước các anchor 3D rồi dự đoán offset và class cho mỗi anchor (như phương pháp PIXOR, AVOD).

Metric học phù hợp cho định vị: Thay vì Smooth L1 thuần túy, có thể dùng **IoU Loss** hoặc các loss trực tiếp trên không gian 3D để mô hình học ra hộp khớp hơn. Cũng nên áp dụng **Non-Maximum Suppression (NMS)** trên các đề xuất hộp để loại bỏ trùng lặp.

- **Đánh giá trên KITTI benchmark:** Huấn luyện và tinh chỉnh mô hình trên tập train KITTI, sau đó **đánh giá kết quả trên server benchmark KITTI** để có so sánh khách quan với các phương pháp khác. Mục tiêu là cải thiện dần chỉ số mAP 3D của mô hình tiệm cận với các mô hình hiện có.
- **Xây dựng ứng dụng trực quan và API:** Song song nghiên cứu thuật toán, việc **phát triển một ứng dụng demo** sẽ được thực hiện. Trước mắt, có thể xây dựng một **web demo** nhỏ cho phép upload file point cloud và chạy mô hình đã huấn luyện để xem kết quả (sử dụng các thư viện như Open3D để hiển thị). Về lâu dài, hướng đến xây dựng một **API dịch vụ** cho phép gửi dữ liệu 3D lên server và nhận về kết quả phân tích (phân lớp/phân đoạn/phát hiện), hỗ trợ cho các ứng dụng khác trong lĩnh vực xe tự hành hoặc robotics. Điều này giúp **đưa mô hình vào thực tiễn**, đồng thời tạo tiện ích để tiếp tục thu thập phản hồi và dữ liệu thực nhằm cải tiến mô hình.

Tóm lại, những nghiên cứu và triển khai trong luận văn này có thể xem là bước khởi đầu. Mô hình PointNet++ đã chứng minh hiệu quả trên dữ liệu KITTI ở mức độ nhất định; nhưng vẫn còn nhiều không gian để **tối ưu và mở rộng**. Hướng phát triển trong tương lai sẽ tập trung vào **nâng cao độ chính xác** (qua cải tiến kiến trúc và huấn luyện) và **mở rộng tính ứng dụng** (qua phát triển hệ thống phát hiện hoàn chỉnh và giao diện sử dụng). Những bước đi này sẽ đưa mô hình tiến gần hơn đến ứng dụng thực tế, đồng thời đóng góp vào sự phát triển chung của lĩnh vực xử lý point cloud 3D.

TÀI LIỆU THAM KHẢO

- [1] Voxel51, "A Comprehensive Guide to Point Cloud Data," Voxel51 Blog. [Online]. Available: <https://voxel51.com/blog/comprehensive-guide-point-cloud-data>. [Accessed: 28-07-2025].
- [2] VnEconomy, "LiDAR là gì và tại sao lại quan trọng đối với xe tự hành," VnEconomy. [Online]. Available: <https://vneconomy.vn/automotive/lidar-la-gi-va-tai-sao-lai-quan-trong-doi-voi-xe-tu-hanh.htm>. [Accessed: 28-07-2025].
- [3] Open3D, "Point cloud," Open3D Documentation. [Online]. Available: <https://www.open3d.org/docs/0.9.0/tutorial/Basic/pointcloud.html>. [Accessed: 28-07-2025].
- [4] Wikipedia, "Point cloud," Wikipedia, the free encyclopedia. [Online]. Available: https://en.wikipedia.org/wiki/Point_cloud. [Accessed: 28-07-2025].
- [5] [fnmCristiano \surSaltori], "[Novel class discovery meets foundation models for 3D semantic segmentation]," arXiv preprint arXiv:2312.03782, 2023. [Online]. Available: <https://arxiv.org/html/2312.03782v2>. [Accessed: 28-07-2025].
- [6] Think Autonomous, "Voxel vs Points," Think Autonomous Blog. [Online]. Available: <https://www.thinkautonomous.ai/blog/voxel-vs-points/>. [Accessed: 28-07-2025].
- [7] N. Malviya, "PointNet: A Point-Based Architecture for 3D Point Cloud Data," Medium. [Online]. Available: <https://medium.com/@nikitamalviya/pointnet-a-point-based-architecture-for-3d-point-cloud-data-32c1e7a16b84>. [Accessed: 28-07-2025].

- [8] Apple Machine Learning Research, "VoxelNet: End-to-End Learning for Point Cloud Based 3D Object Detection," Apple Machine Learning Research. [Online]. Available: <https://machinelearning.apple.com/research/voxelnet-end-to-end>. [Accessed: 28-07-2025].
- [9] Stanford CS230, "Confusion Matrix," Stanford CS230 Deep Learning. [Online]. Available: <https://cs230.stanford.edu/section/8/>. [Accessed: 28-07-2025].
- [10] V7 Labs, "Mean Average Precision (mAP)," V7 Labs Blog. [Online]. Available: <https://www.v7labs.com/blog/mean-average-precision>. [Accessed: 28-07-2025].
- [11] Dathop, "LiDAR là gì? LiDAR hoạt động như thế nào?," Dathop. [Online]. Available: <https://dathop.com.vn/post/lidar-la-gi-lidar-hoat-dong-nhu-the-nao.html>. [Accessed: 28-07-2025].
- [12] VMTS, "Đám mây điểm (Point Cloud)," VMTS. [Online]. Available: <https://vmts.ch/vi/dam-may-diem-point-cloud>. [Accessed: 28-07-2025].
- [13] SemanticKITTI, "SemanticKITTI Dataset," SemanticKITTI. [Online]. Available: <https://semantic-kitti.org/>. [Accessed: 28-07-2025].
- [14] Neuvition, "Solid State LiDAR," Neuvition. [Online]. Available: <https://www.neuvition.com/vi/media/solid-state-lidar-5.html>. [Accessed: 28-07-2025].
- [15] ZTE Corporation, "ZTE Official Website," ZTE. [Online]. Available: <https://www.zte.com.cn/china/index.html>. [Accessed: 28-07-2025].
- [16] D. Doria, "ClusteringSegmentation," GitHub repository. [Online]. Available: <https://github.com/daviddoria/ClusteringSegmentation>. [Accessed: 28-07-2025].

- [17] Papers with code, “Papers with code” GitHub repository. [Online]. Available: <https://github.com/paperswithcode>. [Accessed: 28-07-2025].